GUÍA COMPLETA Y AMPLIADA · A4 PARA IMPRIMIR

Ingeniería y Calidad de Software

Guía completa para el segundo parcial

UNIDAD 2

Gestión Lean Agile de Productos de Software

UNIDAD 3

Gestión de software como producto

UNIDAD 4

Aseguramiento de Calidad de Proceso y Producto

Desarrollo ampliado, casos, fórmulas, cuadros, preguntas y resúmenes para el parcial.

Basado en el material de la cátedra UTN-FRC y la bibliografía compartida en el proyecto. Versión ampliada: reúne todos los temas de las respuestas previas, con explicaciones más detalladas y ejemplos. No es una transcripción literal.

Octubre de 2026 · Formato A4

Contenido

Índice de secciones · Los números indican la página de inicio.

UNIDAD 2 · GESTIÓN LEAN AGILE DE PRODUCTOS DE SOFTWARE 3

1. Filosofía Lean 3
2. Método Kanban en el desarrollo de software 5
3. Métricas de Kanban y gestión del flujo 7
4. Métricas en enfoques Tradicional, Lean y Agile 8
5. Desarrollo ampliado de Lean: decisiones, aprendizaje, calidad y ejemplos 9
6. Kanban en profundidad: diseño, operación y mejora del servicio 12
7. Métricas: problemas resueltos, lectura de gráficos y límites de interpretación 13

Resumen para el parcial · Unidad 2 15

UNIDAD 3 · GESTIÓN DE SOFTWARE COMO PRODUCTO 17

1. Prácticas continuas 17
2. Integración continua (Continuous Integration - CI) 17
3. Construcción continua y tipos de Build 18
4. Pruebas continuas (Continuous Testing) 19
5. Entrega continua y despliegue continuo 20
6. Prácticas complementarias y ejemplo integrador 21
7. Prácticas continuas: desarrollo detallado, criterios y problemas habituales 22

Resumen para el parcial · Unidad 3 25

UNIDAD 4 · ASEGURAMIENTO DE CALIDAD DE PROCESO Y PRODUCTO 26

1. Conceptos generales sobre calidad 26
2. Calidad de producto y calidad de proceso 27
3. Importancia de trabajar para y con calidad 29
4. QA, planificación, QC, testing, verificación y validación 30
5. Actividades relacionadas con SQA 31
6. Principales modelos de calidad: CMMI, SPICE e ISO 31
7. Auditorías de software 34
8. Calidad de producto: planificación de pruebas 36
9. Niveles de prueba 38
10. Tipos de pruebas 39
11. Técnicas de prueba 39
12. Herramientas para probar software 41
13. Los siete principios generales de testing 41
14. Revisiones técnicas de software 41
15. Testing en ambientes ágiles 44
16. Desarrollo ampliado: razonamiento de calidad de proceso y de producto 47
17. Profundización de modelos: qué evalúan y qué no evalúan 49
18. Auditorías en profundidad: evidencia, roles y cierre de hallazgos 50
19. Planificación de testing y diseño: casos completos 52
20. Inspecciones y revisiones técnicas: desarrollo detallado 54
21. Testing ágil: mapa completo del cambio de enfoque 55
22. Guía de repaso por preguntas: Unidad 4 57

Resumen para el parcial · Unidad 4 58

MAPA DE COBERTURA PARA ESTUDIAR SIN DEJAR TEMAS FUERA 60

Bibliografía de referencia 62

UNIDAD 2 · GESTIÓN LEAN AGILE DE PRODUCTOS DE SOFTWARE

1. Filosofía Lean

1.1. Concepto y origen

Lean es una filosofía de gestión que busca maximizar el valor entregado al cliente y minimizar el desperdicio. Sus antecedentes se encuentran en el Sistema de Producción de Toyota, que promueve la mejora continua, la eliminación de actividades que no aportan valor, el respeto por las personas y la organización del trabajo para crear un flujo eficiente. El pensamiento Lean no consiste en pedir que las personas trabajen más rápido, sino en mejorar el sistema completo de trabajo.

En la fabricación industrial se analizan inventarios, tiempos de espera, movimientos innecesarios y defectos. En software, el producto es fundamentalmente información y conocimiento, por lo que los desperdicios aparecen como trabajo parcialmente terminado, transferencias, esperas, funcionalidades innecesarias y errores descubiertos tarde. Esta diferencia hace que Lean no pueda trasladarse mecánicamente de una fábrica a un equipo de desarrollo.

Valor: aquello que resulta útil para quien recibe el producto o servicio. No se identifica necesariamente con producir más funcionalidades. Una característica adicional que nadie necesita puede representar esfuerzo y complejidad sin generar valor. Para identificar el valor se deben comprender tanto las necesidades explícitas como las expectativas reales de los usuarios.

Just in Time (JIT): producir o realizar trabajo cuando se necesita y en cantidades apropiadas, evitando acumular inventario o tareas que todavía no pueden utilizarse. En software se relaciona con la reducción de trabajo en progreso y con completar funcionalidades valiosas en lugar de comenzar muchas simultáneamente.

1.2. Los siete principios de Lean Software Development

Mary y Tom Poppendieck presentan siete principios de Lean aplicados al desarrollo de software. Es importante conocerlos, explicarlos y relacionarlos con ejemplos concretos.

1. Eliminar desperdicios (Eliminate Waste). Identificar y eliminar actividades que consumen tiempo o recursos sin aportar valor al cliente. Ejemplo: evitar desarrollar un sistema de reportes complejo que ningún usuario ha solicitado o validado.

2. Amplificar el aprendizaje (Amplify Learning). El desarrollo de software es una actividad de aprendizaje y resolución de incertidumbre. Iteraciones cortas, pruebas, integración frecuente, prototipos y retroalimentación permiten descubrir errores de comprensión tempranamente. Ejemplo: presentar una versión simple de una funcionalidad antes de invertir semanas en implementarla por completo.

3. Decidir lo más tarde responsablemente posible (Decide as Late as Possible). No tomar decisiones irreversibles antes de contar con información suficiente, siempre que postergarlas no implique perder opciones importantes. No significa procrastinar. Ejemplo: mantener abiertas dos alternativas de integración mientras se validan sus riesgos técnicos.

4. Entregar tan rápido como sea posible (Deliver as Fast as Possible). Reducir el tiempo desde la identificación de una necesidad hasta la obtención de una solución útil. La entrega rápida favorece el

aprendizaje y disminuye el riesgo de invertir demasiado en decisiones incorrectas. No significa sacrificar calidad ni trabajar sin límites.

5. Empoderar al equipo (Empower the Team). Las personas que realizan el trabajo poseen conocimiento valioso sobre cómo mejorarlo. La organización debe facilitar autonomía, aprendizaje, responsabilidad compartida y capacidad de decisión. El respeto por las personas es una parte central del enfoque.

6. Construir la integridad desde dentro (Build Integrity In). Incorporar la calidad y coherencia del producto durante todo el desarrollo, no al final. Comprende pruebas tempranas, diseño adecuado, refactorización, automatización y criterios claros de aceptación. La integridad conceptual significa que el sistema presenta un diseño coherente; la integridad percibida se relaciona con la experiencia del usuario.

7. Ver el todo (See the Whole). Optimizar el sistema completo y no únicamente la productividad de un individuo o sector. Un equipo puede programar rápidamente y, sin embargo, producir poca entrega útil si el trabajo se acumula esperando revisión, pruebas o despliegue.

Ejemplo integrador: si el equipo comienza diez historias y termina solo dos, su alto volumen de actividad no implica alto valor entregado. Limitar el trabajo simultáneo, reducir esperas y completar historias de extremo a extremo puede mejorar los resultados del sistema.

1.3. Los siete desperdicios del desarrollo de software

Los Poppendieck adaptan al software los tipos de desperdicio observados en Lean:

Desperdicio	Significado en software	Ejemplo
Trabajo parcialmente terminado	Inventario de trabajo que todavía no aporta valor.	Historias programadas pero sin integrar ni probar.
Procesos adicionales	Actividades administrativas o aprobaciones sin beneficio suficiente.	Firmas repetidas que no mejoran las decisiones.
Funcionalidades adicionales	Características que no responden a necesidades relevantes.	Opciones sofisticadas que nadie utiliza.
Cambio de tareas (task switching)	Pérdida de foco y esfuerzo al alternar trabajos.	Un programador atiende cinco historias simultáneamente.
Esperas	Tiempo en el que el trabajo no avanza.	Esperar días por una aprobación o un ambiente de testing.
Movimiento o transferencias	Esfuerzo por pasar información entre personas o áreas.	Repetir explicaciones por documentación ambigua y muchos handoffs.
Defectos	Errores, retrabajo y fallas que consumen capacidad.	Una regla mal definida que requiere corregir varias capas del sistema.

Los desperdicios están relacionados entre sí. Por ejemplo, demasiado trabajo en progreso puede aumentar cambios de contexto, espera y defectos. Por eso es importante analizar las causas del problema y no solamente sus síntomas.

1.4. Mapeo de flujo de valor (Value Stream Mapping)

El **Value Stream Mapping (VSM)** permite visualizar las actividades necesarias para transformar una necesidad en un producto entregado. El objetivo es identificar pasos, tiempos de procesamiento, esperas, transferencias y desperdicios.

Por ejemplo, una funcionalidad puede tardar dos horas en análisis, ocho en programación, cuatro en pruebas y dos en despliegue: **16 horas de trabajo efectivo**. Si permanece 40 horas esperando asignación y otras 24 esperando revisión, el tiempo total de flujo será **80 horas**. La eficiencia del flujo sería 16/80 = **20 %**. Este ejemplo es didáctico: no corresponde a una medición de la bibliografía.

La intervención Lean debería orientarse especialmente a las esperas, que representan la mayor parte del tiempo. Agregar más horas de programación no resolvería el principal cuello de botella.

2. Método Kanban en el desarrollo de software

2.1. ¿Qué es Kanban?

Kanban es un método para gestionar y mejorar servicios de trabajo del conocimiento mediante la visualización, la limitación del trabajo en progreso y la mejora evolutiva del flujo. Se utiliza para mejorar la manera en que el trabajo atraviesa un proceso existente. No exige reemplazar de golpe la estructura organizacional ni definir roles idénticos a los de Scrum.

Kanban, Lean y Agile: Lean es una filosofía de gestión; Kanban proporciona un método de evolución y gestión del flujo; Agile comprende valores y enfoques de desarrollo adaptativos. Pueden combinarse. Kanban no es simplemente un tablero de columnas ni equivale a Scrum.

En un tablero típico aparecen estados como Pendiente, Análisis, Desarrollo, Pruebas y Terminado. Las tarjetas representan elementos de trabajo; las columnas muestran su estado. El tablero debe reflejar el flujo real, incluidas las esperas relevantes.

2.2. Principios de gestión del cambio y de prestación de servicios

La bibliografía de Anderson y Carmichael plantea seis principios fundacionales, en dos conjuntos de tres.

Principios de gestión del cambio: comenzar con lo que se hace actualmente; acordar la mejora mediante cambios evolutivos; fomentar actos de liderazgo en todos los niveles. La meta es generar mejoras sostenibles sin depender de una reorganización disruptiva.

Principios de prestación de servicios: comprender las necesidades y expectativas de los clientes; gestionar el trabajo y permitir que las personas se autoorganicen alrededor de él; revisar periódicamente la red de servicios y sus políticas para mejorar resultados.

La guía condensada presenta además tres principios orientadores: **servicio al cliente, gestión del trabajo y evolución del sistema de trabajo**, que sintetizan la orientación del método.

2.3. Valores de Kanban

La literatura de Kanban presenta nueve valores: **transparencia, equilibrio, colaboración, foco en el cliente, flujo, liderazgo, entendimiento, acuerdo y respeto**. No son prácticas operativas aisladas, sino principios culturales que facilitan la evolución del servicio. Por ejemplo, la transparencia es necesaria para visualizar bloqueos; la colaboración permite resolverlos; el respeto favorece que la mejora no se transforme en una búsqueda de culpables.

2.4. Las seis prácticas generales de Kanban

1. Visualizar el trabajo. Representar tipos de trabajo, estados, bloqueos, reglas y dependencias para comprender qué sucede. Un tablero permite observar dónde se acumula el trabajo y cuál es el siguiente paso.

2. Limitar el trabajo en progreso (WIP). Establecer límites sobre la cantidad de elementos que pueden encontrarse simultáneamente en ciertos estados. Si la columna Desarrollo tiene límite 3 y ya contiene tres tareas, no debería comenzarse otra sin liberar capacidad o modificar explícitamente la política.

3. Gestionar el flujo. Analizar cómo atraviesan el proceso las solicitudes, identificar cuellos de botella y mejorar la previsibilidad y el tiempo de entrega.

4. Hacer explícitas las políticas. Establecer reglas visibles sobre entrada y salida de columnas, prioridad, clases de servicio, condiciones de bloqueo y criterios de aceptación.

5. Implementar ciclos de retroalimentación. Realizar reuniones y revisiones que permitan coordinar el servicio, analizar métricas, reponer trabajo y mejorar prácticas.

6. Mejorar colaborativamente y evolucionar mediante experimentos. Plantear hipótesis de mejora, aplicar cambios controlados, medir los resultados y adaptar el sistema.

2.5. Sistema push y sistema pull

En un sistema **push**, se asigna trabajo a la siguiente actividad aun cuando ese sector no disponga de capacidad. Esto puede generar acumulación de tareas y largas esperas. En un sistema **pull**, un elemento avanza cuando existe capacidad disponible y se cumplen las políticas para admitirlo. Los límites WIP ayudan a operacionalizar el pull.

Ejemplo: si las pruebas están saturadas, programar todavía más funcionalidades puede aumentar el inventario sin generar entregas. El equipo podría concentrarse en terminar y verificar los elementos ya desarrollados antes de comenzar otros.

2.6. Punto de compromiso, punto de entrega y tiempo de flujo

El **punto de compromiso** es el momento a partir del cual la organización asume el compromiso de atender un elemento de trabajo. El **punto de entrega** es el punto donde ese elemento se considera entregado según la definición del servicio. Entre ambos se mide el tiempo que el sistema tarda en cumplir el compromiso.

La terminología de *lead time* y *cycle time* varía entre autores y herramientas. Para evitar ambigüedades, Kanban recomienda definir explícitamente los puntos de inicio y finalización de cada métrica. En la guía oficial, el **tiempo de entrega del elemento** se mide desde su entrada al sistema considerado hasta su salida. No debe compararse un valor con otro si ambos se miden entre puntos diferentes.

2.7. Políticas explícitas y clases de servicio

Las políticas pueden establecer el WIP de cada columna, las condiciones para considerar una actividad terminada, la prioridad de las solicitudes, el tratamiento de elementos bloqueados y los acuerdos de servicio.

Las **clases de servicio** permiten diferenciar el tratamiento de solicitudes según su riesgo, urgencia o costo de demora. En la bibliografía de Anderson aparecen ejemplos como *expedite* (urgente), *fixed date* (fecha fija), estándar e intangible. No conviene convertir todo en urgente: una política de expedición mal utilizada interrumpe el flujo ordinario.

2.8. STATIK

STATIK (*Systems Thinking Approach to Introducing Kanban*) es un enfoque para introducir Kanban comprendiendo primero el sistema de trabajo. La bibliografía propone analizar propósito y fuentes de insatisfacción, demandas y capacidades, tipos de trabajo, flujo actual, clases de servicio, diseño del sistema, reuniones y mecanismos de mejora. La idea es que el tablero no se invente de manera descontextualizada: debe representar un servicio real y sus problemas.

2.9. Cadencias de Kanban

Kanban propone ciclos regulares de coordinación y retroalimentación. Entre los más conocidos se encuentran reunión Kanban diaria, reposición (*replenishment*), revisión de entrega, revisión del servicio, revisión de operaciones, revisión de riesgos y revisión estratégica. No todas las organizaciones necesitan todas las cadencias ni deben aplicar una duración fija. Su frecuencia depende de la naturaleza del servicio y del tipo de decisión.

3. Métricas de Kanban y gestión del flujo

3.1. Lead time / tiempo de entrega

Mide el tiempo transcurrido desde un punto de inicio acordado hasta un punto de finalización acordado. Permite estimar cuánto debe esperar un cliente por un elemento de trabajo. Es importante comunicar el alcance exacto: por ejemplo, desde que se compromete la solicitud hasta que está disponible en producción.

3.2. Cycle time / tiempo de ciclo

En muchas herramientas, el tiempo de ciclo comienza cuando se inicia el trabajo activo. La distinción respecto del lead time no es universal, por lo que deben definirse los límites de medición. Si una solicitud espera 3 días antes de comenzar y luego tarda 4 en procesarse, podría registrarse lead time de 7 días y cycle time de 4, según las políticas adoptadas.

3.3. Throughput / tasa de entrega

Es la cantidad de elementos completados por unidad de tiempo. Ejemplo: ocho historias terminadas por semana. Debe cuidarse la definición del elemento: contar tareas de tamaños muy diferentes puede dificultar comparaciones.

3.4. WIP / trabajo en progreso

Es la cantidad de elementos que se encuentran dentro de un sistema de trabajo sin haber sido completados. El WIP elevado tiende a aumentar la espera, el cambio de contexto y la necesidad de coordinación. Reducirlo puede facilitar la detección de bloqueos y el enfoque en terminar trabajo.

3.5. Ley de Little

En condiciones apropiadas de estabilidad del flujo, la **Ley de Little** relaciona tres magnitudes:

WIP promedio = Throughput promedio × Tiempo de flujo promedio.

Si un sistema mantiene, en promedio, 12 elementos en progreso y finaliza 3 elementos por día, el tiempo promedio sería 12/3 = **4 días**. La fórmula describe una relación entre promedios y requiere que las medidas correspondan al mismo sistema y período apropiado. No debe interpretarse como una predicción exacta para cada elemento.

3.6. Otras métricas útiles

Edad del trabajo en progreso (Work Item Age). Tiempo que un elemento abierto lleva en el sistema. Permite identificar tareas que se aproximan a umbrales de riesgo.

Eficiencia del flujo. Relación entre tiempo de trabajo efectivo y tiempo total de flujo. Una eficiencia baja puede indicar esperas relevantes, pero su interpretación exige comprender el tipo de trabajo.

Bloqueos. Cantidad, frecuencia y duración de los impedimentos. Ayuda a detectar dependencias o políticas que dificultan el flujo.

Fiabilidad de entrega. Evaluación de la probabilidad de cumplir una fecha o un acuerdo de nivel de servicio. Conviene utilizar distribuciones observadas, no solamente promedios.

Carga de fallas y calidad. Reprocesos y defectos que consumen capacidad en lugar de generar nuevas entregas.

Costo de demora (Cost of Delay). Consecuencias económicas o de valor de postergar un trabajo. Permite priorizar solicitudes según el impacto de esperar.

3.7. Gráficos de Kanban

Diagrama de flujo acumulado (CFD). Representa la cantidad acumulada de elementos en cada estado a lo largo del tiempo. Permite visualizar acumulaciones y cambios en el flujo. Si una banda se ensancha sostenidamente, puede existir una acumulación en ese estado.

Histograma de tiempos de entrega. Muestra la distribución de duraciones históricas: permite observar dispersión, valores típicos y casos extremos.

Gráfico de dispersión de tiempos de entrega. Cada punto representa un elemento terminado y su duración; ayuda a detectar variabilidad y cambios en el desempeño.

Previsiones probabilísticas y Monte Carlo. Utilizan observaciones y simulaciones para analizar rangos de fechas o cantidades de entrega. No prometen certezas; permiten hacer explícita la incertidumbre.

4. Métricas en enfoques Tradicional, Lean y Agile

4.1. Enfoque tradicional

Se concentra frecuentemente en controlar el cumplimiento del plan y de las restricciones de plazo, presupuesto y alcance. Las métricas típicas incluyen esfuerzo real frente a planificado, desviación de costos, cumplimiento de hitos, avance de tareas, defectos y utilización de recursos.

Como complemento conceptual —no desarrollado exhaustivamente en la bibliografía proporcionada— puede utilizarse la **Gestión del Valor Ganado (EVM)**: PV (valor planificado), EV (valor ganado), AC (costo real); CV = EV - AC; SV = EV - PV; CPI = EV/AC; SPI = EV/PV. Estas fórmulas sirven para analizar variaciones en costo y cronograma, pero no expresan por sí mismas el valor que recibe el usuario.

4.2. Enfoque Lean / Kanban

Se concentra en el **flujo de valor**, la cantidad de trabajo iniciado, el tiempo hasta terminarlo y la variabilidad. Sus métricas características son WIP, throughput, tiempo de entrega, tiempo de ciclo, eficiencia del flujo, edad de los elementos, bloqueos y previsibilidad.

La pregunta central cambia de «¿cuánto esfuerzo consumimos?» a «¿cuánto tarda el trabajo en convertirse en valor entregado y qué está impidiendo el flujo?». Optimizar el trabajo ocupado de cada persona no garantiza optimizar el sistema completo.

4.3. Enfoque Agile / Scrum

Utiliza indicadores para inspeccionar el avance de incrementos y adaptar planes. Entre los más conocidos están la **velocidad**, burndown, burnup, historias completadas, cumplimiento del objetivo del Sprint, defectos y capacidad de entregar incrementos probados.

Velocidad: cantidad de puntos de historia u otra unidad de estimación terminados por iteración. Resulta útil para las previsiones del propio equipo; no debe utilizarse para comparar mecánicamente la productividad de equipos diferentes ni para medir el rendimiento de personas individuales.

Burndown: representa trabajo pendiente a lo largo del tiempo. Ayuda a observar si disminuye el trabajo restante, pero puede ocultar cambios de alcance.

Burnup: muestra trabajo completado y alcance total; permite representar mejor el efecto de agregar o quitar trabajo.

Calidad y Done: un incremento no debería contarse como terminado si todavía requiere el testing necesario según los acuerdos del equipo. Las métricas de actividad no sustituyen la inspección real del producto.

4.4. Lean UX: outputs y outcomes

Lean UX, de Gothelf y Seiden, distingue **outputs** (entregables o funcionalidades producidas) de **outcomes** (cambios observables en comportamientos o resultados). Entregar una pantalla no demuestra necesariamente que mejoró la experiencia del usuario. Por ejemplo, aumentar la tasa de finalización de una solicitud puede ser un outcome más relevante que contar formularios desarrollados.

El enfoque promueve establecer hipótesis, experimentar, observar el comportamiento de usuarios y aprender. Las métricas deben acompañar el objetivo del producto y no recompensar solamente la producción de funcionalidades.

4.5. Comparación de enfoques

Aspecto	Tradicional	Lean / Kanban	Agile / Scrum
Foco	Cumplimiento del plan	Flujo y desperdicios	Entrega incremental y adaptación
Pregunta típica	¿Estamos dentro del tiempo y presupuesto?	¿Cuánto tarda en fluir y entregarse el trabajo?	¿Qué incremento terminamos y qué aprendimos?
Métricas	Plazo, costo, avance, esfuerzo	WIP, lead time, throughput, CFD	Velocidad, burndown, burnup, Done
Riesgo de mal uso	Confundir cumplimiento del plan con valor	Optimizar solo métricas del flujo	Convertir velocidad en objetivo o ranking
Complemento necesario	Calidad y valor al usuario	Calidad y necesidades del servicio	Calidad y resultados del producto

5. Desarrollo ampliado de Lean: decisiones, aprendizaje, calidad y ejemplos

5.1. Lean no es solo «hacer más con menos»

En el origen industrial, Lean se relaciona con el **Sistema de Producción Toyota** y la búsqueda de un flujo de creación de valor confiable. En software, no alcanza con medir cuántas horas trabajan las personas, porque

una parte importante del trabajo consiste en descubrir el problema correcto, decidir qué construir y aprender de los resultados. Un equipo puede estar ocupado el 100 % del tiempo y, al mismo tiempo, generar pocas funcionalidades terminadas. La utilización máxima de personas y recursos **no equivale necesariamente** al máximo rendimiento del sistema. Cuando todos los participantes están saturados, pequeñas interrupciones pueden convertirse en esperas largas.

La idea de **Just in Time (JIT)** se relaciona con producir lo que se necesita cuando se necesita. En desarrollo de software se traduce, entre otras cosas, en evitar preparar por adelantado grandes cantidades de trabajo que aún no ha sido validado. **Jidoka**, en su sentido de calidad incorporada al proceso, ayuda a entender por qué conviene detectar problemas en el momento en que se producen y detener o corregir una línea de trabajo defectuosa antes de propagar el problema. No significa que software y manufactura sean idénticos: el trabajo del conocimiento presenta incertidumbre y aprendizaje de manera especialmente intensa.

Valor y desperdicio dependen del contexto. Una prueba de concepto que finalmente se descarta puede haber creado valor si permitió evitar una inversión mucho más costosa en una solución inviable. No debe confundirse «trabajo que no terminó en una característica comercializable» con «trabajo inútil»: aprender puede ser el objetivo legítimo de la actividad.

5.2. Análisis profundo de los siete principios

Eliminar desperdicios: antes de eliminar actividades hay que distinguir su función. Una revisión técnica, aunque no genera directamente una característica visible para el usuario, puede evitar defectos costosos y, por lo tanto, aportar valor indirecto. El objetivo no es quitar todas las tareas de control, sino reducir las que no producen un beneficio justificable. Una pregunta útil es: «¿Qué ocurriría si dejáramos de hacer esta actividad?». Si desaparece una protección importante, probablemente no sea un desperdicio.

Amplificar el aprendizaje: se aprende mediante ciclos de hipótesis, construcción, medición y adaptación. Las historias pequeñas, prototipos y pruebas rápidas limitan cuánto trabajo puede estar basado en suposiciones incorrectas. Por ejemplo, antes de crear un módulo complejo de búsqueda, se pueden comprobar con usuarios tres maneras de presentar los resultados. El aprendizaje temprano reduce incertidumbre y previene desperdicios posteriores.

Decidir lo más tarde responsablemente posible: la palabra «responsablemente» es clave. Conviene postergar decisiones difíciles de revertir mientras se mantienen alternativas disponibles, pero no esperar tanto que se pierda una oportunidad o se bloquee a otros equipos. Si hay que contratar una infraestructura que tarda meses en entregarse, la decisión tendrá una fecha límite diferente de una decisión que puede resolverse con una configuración en pocos minutos.

Entregar tan rápido como sea posible: la velocidad buscada es la del flujo completo de valor, no la velocidad de escribir código. La calidad, el testing y la integración son necesarios para que la entrega sea útil. Reducir el tamaño de los lotes hace más sencilla la detección de errores y la obtención de feedback. Trabajar rápido con muchos defectos puede aumentar, y no reducir, el tiempo total de entrega.

Empoderar al equipo: es un error sustituir la mejora del sistema por presión individual. Los desarrolladores, testers, responsables de producto y operadores tienen conocimiento local sobre restricciones y problemas. El liderazgo debe crear condiciones para que puedan proponer soluciones, colaborar y asumir responsabilidad sobre el resultado.

Construir integridad desde dentro: la calidad debe estar presente desde la definición de necesidades y el diseño. La **integridad percibida** se vincula con la experiencia coherente que recibe el usuario; la **integridad conceptual** con un sistema internamente consistente y comprensible. Una interfaz que parece uniforme pero funciona sobre un diseño frágil puede deteriorarse rápidamente al evolucionar el producto.

Ver el todo: optimizar un sector puede perjudicar a otro. Si el área de desarrollo aumenta su producción de funcionalidades sin mejorar la capacidad de verificación, aumentará la cola de trabajo esperando testing. El resultado global puede ser peor aunque la métrica de productividad de programación haya mejorado. Lean propone gestionar estas interdependencias.

5.3. Ejemplos adicionales de los siete desperdicios

Desperdicio	Señal observable	Consecuencia probable	Posible intervención
Trabajo parcialmente terminado	Muchas historias empezadas y pocas entregadas	Inventario oculto y riesgo de quedar obsoleto	Limitar WIP y terminar antes de empezar
Procesos adicionales	Múltiples aprobaciones duplicadas	Esperas y esfuerzo administrativo	Revisar qué controles agregan valor
Funcionalidades adicionales	Características sin evidencia de necesidad	Complejidad, mantenimiento y costo	Validar hipótesis y priorizar por valor
Cambio de tareas	Una persona cambia de historia varias veces al día	Pérdida de foco y defectos	Reducir multitarea y dependencias
Espera	Trabajo bloqueado por decisión, ambiente o proveedor	Lead time elevado	Hacer visible y resolver el bloqueo
Movimiento/transferencias	La información pasa por muchas áreas	Pérdida de contexto y retrabajo	Equipos colaborativos y comunicación directa
Defectos	Correcciones repetidas y escapes a producción	Reprocesos y deterioro de confianza	Revisiones tempranas, CI y pruebas

Un defecto corregido puede ser la manifestación final de una espera o de una transferencia defectuosa: por ejemplo, una regla del negocio se transmite verbalmente de un sector a otro, se interpreta mal y meses después produce una falla en producción. Por eso, eliminar un desperdicio suele requerir comprender su **causa sistémica**.

5.4. Value Stream Mapping: resolución paso a paso

Supongamos un servicio que recibe solicitudes de cambio. Su flujo tiene cinco actividades: análisis, desarrollo, revisión, testing y despliegue. Los **tiempos activos** observados son 3, 8, 2, 4 y 1 horas respectivamente. Además existen esperas de 12, 18, 10 y 6 horas entre las etapas.

- Tiempo de trabajo efectivo: 3 + 8 + 2 + 4 + 1 = **18 horas**.
- Tiempo de espera: 12 + 18 + 10 + 6 = **46 horas**.
- Tiempo total: 18 + 46 = **64 horas**.
- Eficiencia del flujo ilustrativa: 18 / 64 = **28,125 %**.

La primera reacción podría ser acelerar programación, pero el análisis muestra que el desarrollo activo representa solo 8 horas. La mayor oportunidad está en las **46 horas de espera**. Podrían reducirse con revisión más frecuente, capacidad reservada para testing o políticas explícitas para desbloquear tareas.

Para construir un mapa útil se debe **definir el inicio y final del flujo**, representar las actividades reales —no solamente las del procedimiento formal—, registrar las esperas y sus causas, visualizar transferencias, distinguir valor de desperdicio y proponer experimentos que se puedan evaluar.

6. Kanban en profundidad: diseño, operación y mejora del servicio

6.1. Elementos de un sistema Kanban bien definido

Un tablero aislado no constituye todo el método. El servicio debe tener **tipos de trabajo identificables**, un flujo explícito, límites de WIP donde correspondan, reglas claras de movimiento, criterios para tomar nuevas solicitudes, políticas de priorización y datos que permitan comprender su rendimiento.

Imaginemos un tablero con columnas «Opciones», «Análisis», «Desarrollo», «Validación» y «Entregado». Dentro de la columna «Desarrollo» puede existir un límite WIP de 3; dentro de «Validación», un límite de 2. Cuando Validación llega a su límite, el equipo debe colaborar para completar o desbloquear elementos antes de seguir generando más inventario. El límite no se utiliza para culpar a una persona; su función es **exponer una restricción del flujo**.

Es útil distinguir estados activos y de espera. Una columna «Esperando aprobación» puede hacer evidente una cola que antes no se medía. También conviene mostrar explícitamente elementos bloqueados y registrar su causa y duración.

6.2. Los seis principios fundacionales y su aplicación

Gestión del cambio: comenzar con lo que existe. No se exige reorganizar todos los equipos antes de empezar; se observa cómo trabajan realmente y se hacen visibles las prácticas actuales. **Acordar mejoras evolutivas:** proponer cambios graduales basados en problemas concretos y resultados observados. **Fomentar liderazgo a todos los niveles:** cualquier integrante puede detectar y ayudar a resolver un bloqueo.

Prestación de servicios: comprender al cliente. Un servicio debe responder a necesidades y expectativas del cliente. **Gestionar el trabajo y permitir la autoorganización:** el foco está en cómo circula el trabajo, no en controlar la actividad minuto a minuto de cada individuo. **Revisar la red de servicios:** los resultados de un equipo dependen de otros servicios; la mejora debe tener en cuenta esas interacciones.

6.3. Seis prácticas: qué evidencia debería verse en el trabajo diario

Visualizar: se observan las tarjetas, sus estados, sus bloqueos y las dependencias. **Limitar WIP:** el equipo sabe cuántos elementos puede admitir cada actividad y qué hacer cuando se alcanza el límite. **Gestionar el flujo:** se analizan tiempos de entrega, throughput y acumulaciones, no solo la actividad individual. **Políticas explícitas:** todos saben qué significa terminado, cuándo se puede iniciar una tarea y cómo se gestiona una urgencia. **Bucles de retroalimentación:** las reuniones producen decisiones sobre trabajo, riesgos y mejora. **Mejora evolutiva:** se plantean experimentos y se comprueba su resultado.

6.4. Clases de servicio y costo de demora

Las clases de servicio son políticas para tratar solicitudes con consecuencias de demora diferentes. **Expedite:** solicitudes urgentes que requieren tratamiento excepcional; es importante limitar cuántas pueden interrumpir el flujo. **Fecha fija:** elementos cuyo valor o impacto cambia de forma importante después de determinada fecha. **Estándar:** solicitudes ordinarias con una política regular de atención. **Intangible:** trabajo cuyo beneficio futuro es importante pero el costo inmediato de postergarlo puede no ser visible, como mejoras técnicas preventivas.

El **costo de demora (Cost of Delay)** ayuda a razonar sobre qué se pierde si una solicitud no se termina a tiempo. En una solicitud de fecha fija, el impacto puede aumentar bruscamente cerca de la fecha; en otra, puede crecer gradualmente. No debe confundirse urgencia con importancia: una funcionalidad estratégicamente relevante puede necesitar atención aunque no exista una emergencia visible.

6.5. STATIK: recorrido didáctico

STATIK propone introducir Kanban desde una comprensión sistémica del servicio. Una secuencia didáctica es: (1) identificar para qué existe el servicio y quiénes lo reciben; (2) reconocer fuentes de insatisfacción; (3) analizar los tipos y fuentes de demanda; (4) estudiar la capacidad real de entrega; (5) visualizar el flujo existente; (6) identificar clases de servicio y políticas; (7) diseñar la representación y límites del sistema; y (8) establecer mecanismos de coordinación y mejora.

Ejemplo: un equipo de soporte descubre que recibe 30 incidencias semanales, que la mayoría espera varios días antes de ser analizada y que los incidentes urgentes interrumpen constantemente el trabajo ordinario. Antes de diseñar el tablero, necesita distinguir demanda urgente, mantenimiento, incidencias habituales y mejoras. Después puede establecer clases de servicio y reglas para proteger la capacidad del equipo.

6.6. Cadencias y finalidad

Cadencia	Pregunta que ayuda a responder
Reunión Kanban diaria	¿Qué impide que avance el trabajo comprometido hoy?
Reposición (replenishment)	¿Qué solicitudes deberíamos admitir al sistema y con qué prioridades?
Revisión de entrega (delivery planning)	¿Qué entregas próximas requieren coordinación y gestión de riesgos?
Revisión del servicio	¿Nuestro servicio cumple las expectativas del cliente?
Revisión de operaciones	¿Cómo interactúan los servicios y dónde aparecen dependencias sistémicas?
Revisión de riesgos	¿Qué patrones de riesgo o bloqueo debemos tratar?
Revisión estratégica	¿La capacidad y los servicios siguen alineados con las necesidades futuras?

Kanban no impone aplicar todas las reuniones con idéntica duración. Se eligen las que faciliten decisiones útiles y feedback en el contexto real.

7. Métricas: problemas resueltos, lectura de gráficos y límites de interpretación

7.1. Ejercicio completo de flujo

Un equipo completa durante una semana seis elementos. Sus tiempos de entrega desde el compromiso hasta la entrega son **2, 3, 4, 4, 7 y 10 días**.

- **Throughput de la semana:** 6 elementos por semana.
- **Promedio de tiempo de entrega:** (2 + 3 + 4 + 4 + 7 + 10) / 6 = **5 días**.
- **Mediana:** promedio de los dos valores centrales, (4 + 4) / 2 = **4 días**.
- **Mayor tiempo: 10 días**, relevante para investigar demoras excepcionales.

La diferencia entre mediana y promedio muestra cómo algunos elementos lentos pueden influir en la media. Para compromisos de entrega suele ser importante analizar la distribución completa y utilizar porcentajes de cumplimiento, no solamente un promedio.

Un elemento que sigue abierto desde hace 9 días tiene **Work Item Age de 9 días**; no tiene todavía un tiempo de entrega final porque no salió del sistema. Esta diferencia permite intervenir antes de que un problema se convierta en una demora consumada.

7.2. Ley de Little: interpretación correcta

La relación **WIP promedio = throughput promedio × tiempo de flujo promedio** se aplica a un sistema correctamente delimitado y en condiciones apropiadas de estabilidad. Si, a modo de ejemplo, el WIP medio es 18 y el equipo termina 6 elementos por semana, el tiempo medio compatible con esos datos sería 18/6 = **3 semanas**.

No se debe afirmar que limitar el WIP a 9 hará exactamente que cualquier solicitud tarde una semana y media. La ley describe **promedios del sistema** y su efecto práctico depende también de capacidad, variabilidad, bloqueos y políticas. Reducir WIP sin atender problemas reales puede no producir el resultado esperado. La combinación de métricas importa más que un número aislado.

7.3. CFD o diagrama de flujo acumulado

Un CFD presenta bandas acumuladas por estado a través del tiempo. La **distancia vertical** entre los límites de una banda representa cantidad de trabajo en ese estado o conjunto de estados; la **distancia horizontal** puede ayudar a estimar tiempos de permanencia bajo las interpretaciones y escalas adecuadas. Cuando una banda se ensancha persistentemente, existe un posible crecimiento de inventario o cola. Una pendiente de salidas menor que la de entradas durante un período prolongado indica que entra más trabajo del que se completa.

El CFD no identifica por sí solo la causa del problema. Una banda puede ensancharse por bloqueos, políticas de prioridad, variaciones en capacidad o por elementos de tamaños muy diferentes. Los datos deben complementarse con observación del trabajo.

7.4. Scatterplot, histograma y SLE

Un **scatterplot de tiempos de entrega** ubica los elementos terminados por fecha y duración. Permite observar valores extremos y cambios en el comportamiento. Un **histograma** agrupa duraciones en intervalos y facilita reconocer su distribución. El **Service Level Expectation (SLE)** es una expectativa de tiempo basada en el desempeño histórico y una probabilidad: por ejemplo, «el 85 % de los elementos comparables se completó en ocho días o menos». No significa prometer que absolutamente todos estarán listos en ocho días.

Para elaborar un SLE confiable conviene usar datos comparables, un punto de inicio y fin definido, un período representativo y suficiente información histórica. Un SLE no equivale a una garantía contractual universal.

7.5. Comparar Traditional, Lean y Agile sin confundir objetivos

Tradicional: suele enfatizar cumplimiento de plan, presupuesto y fechas. Ejemplos: desvío de cronograma, desvío de costo, avance respecto de línea base y valor ganado (EVM). **Lean/Kanban:** analiza cómo se mueve el trabajo y qué impide entregar valor: WIP, lead time, cycle time, throughput, bloqueos, eficiencia del flujo y distribución de duraciones. **Agile/Scrum:** puede medir velocidad, burndown, burnup, capacidad, defectos, características probadas y valor entregado.

Estas métricas **no son mutuamente excluyentes**. Un proyecto Scrum puede analizar lead time; un servicio Kanban puede tener restricciones de presupuesto; una organización tradicional puede medir defectos. La diferencia es la perspectiva con que se utilizan para tomar decisiones.

Velocidad: puntos de historia aceptados durante un Sprint. No es una medida universal de productividad entre equipos; los puntos son relativos y dependen de la manera de estimar de cada grupo. Si un equipo termina 21, 25 y 23 puntos en tres Sprints, su promedio histórico es 23 puntos por Sprint; no se deduce de allí que otro equipo con 40 sea más productivo.

Burndown: trabajo restante contra tiempo. Ayuda a ver si el equipo está terminando el trabajo comprometido. **Burnup:** muestra lo completado y el alcance total; permite visualizar mejor cambios de alcance. **EVM:** PV (valor planificado), EV (valor ganado), AC (costo real), SV = EV - PV, CV = EV - AC, SPI = EV / PV y CPI = EV / AC. Estas expresiones requieren criterios consistentes para determinar el avance ganado y tienen restricciones de interpretación en proyectos con alcance evolutivo.

Running Tested Features (RTF): cantidad de funcionalidades terminadas y verificadas de forma útil, no solo código escrito. Como todas las métricas, requiere una definición explícita de «terminada» y «probada».

7.6. Ejemplo EVM y comparación con flujo

Supongamos valores ilustrativos expresados en la misma unidad monetaria: PV = 100, EV = 80 y AC = 90. Entonces **SV = -20** (se ganó menos valor que el planificado), **CV = -10** (se gastó más de lo correspondiente al valor ganado), **SPI = 0,8** y **CPI ≈ 0,89**. En la interpretación tradicional, los indicadores muestran una situación desfavorable respecto de la línea base.

El equipo podría complementar ese análisis con datos de flujo: ¿hay muchas tareas esperando pruebas?, ¿el WIP está creciendo?, ¿los tiempos de entrega son impredecibles? Las métricas Lean permiten investigar el mecanismo por el cual se producen demoras que el indicador financiero refleja, pero no explica.

7.7. Preguntas de razonamiento típicas del examen

¿Por qué reducir WIP puede mejorar la entrega? Porque el equipo concentra la atención en completar trabajo, reduce cambios de contexto y hace visibles colas. La mejora depende de las restricciones reales, por lo que debe medirse.

¿El throughput es lo mismo que la velocidad? No. El throughput cuenta elementos completados por período; la velocidad suele sumar estimaciones relativas —por ejemplo, puntos de historia aceptados por Sprint—. No deben intercambiarse sin definición.

¿La Ley de Little predice el tiempo exacto de cada historia? No. Vincula promedios del sistema bajo condiciones apropiadas.

¿Optimizar la ocupación de cada persona maximiza el flujo? No necesariamente; puede aumentar colas, bloqueos y esperas de trabajo.

¿Kanban obliga a prescindir de iteraciones? No. Puede combinarse con cadencias e iteraciones existentes. El método no exige adoptar los roles de Scrum ni reemplazar de golpe el proceso actual.

¿«Hacer lo más rápido posible» significa entregar sin testing? No. Un producto defectuoso puede producir retrabajo y disminuir el valor realmente entregado.

Resumen para el parcial · Unidad 2

Lean: maximizar valor y eliminar desperdicio. Siete principios de los Poppendieck: eliminar desperdicios, amplificar aprendizaje, decidir lo más tarde responsablemente posible, entregar rápido, empoderar al equipo, construir integridad desde dentro y ver el todo. Siete desperdicios: trabajo parcialmente terminado, procesos extra, funcionalidades extra, cambio de tareas, esperas, transferencias y defectos.

Kanban: método evolutivo para gestionar servicios y flujo de trabajo. Seis prácticas: visualizar, limitar WIP, gestionar flujo, hacer políticas explícitas, retroalimentar y mejorar experimentalmente. El sistema pull evita ingresar trabajo cuando no hay capacidad. Conocer puntos de compromiso y entrega, clases de servicio y STATIK.

Métricas Lean: WIP, throughput, tiempo de flujo, bloqueos, edad y previsibilidad. Ley de Little: WIP = throughput × tiempo promedio. **Tradicional:** costos, plazos, esfuerzo y alcance. **Agile:** velocidad, burndown/burnup y cumplimiento de Done. **Lean UX:** outcomes sobre outputs. Ninguna métrica aislada garantiza calidad ni valor al cliente.

Referencias clave: Poppendieck y Poppendieck, *Lean Software Development: An Agile Toolkit* e *Implementing Lean Software Development*; Anderson y Carmichael, *Kanban Esencial Condensado*; *Guía oficial del Método Kanban*; Anderson, *Kanban: Cambio Evolutivo Exitoso*; Cohn, *Agile Estimating and Planning*; Gothelf y Seiden, *Lean UX*; Kniberg y Skarin, *Kanban y Scrum*.

UNIDAD 3 · GESTIÓN DE SOFTWARE COMO PRODUCTO

1. Prácticas continuas

1.1. Concepto y relación con el producto

Las **prácticas continuas** son actividades de ingeniería y gestión que buscan integrar cambios, comprobar su calidad, preparar entregas y obtener retroalimentación de manera frecuente, sistemática y, cuando es apropiado, automatizada. Contribuyen a que el software evolucione como producto mantenido a lo largo del tiempo, y no como una única entrega final.

La gestión como producto contempla nuevas funcionalidades, correcciones, mantenimiento, evolución tecnológica, seguridad, necesidades cambiantes y feedback de los usuarios. Las prácticas continuas buscan reducir el intervalo entre una modificación y la obtención de información sobre su calidad y valor.

En un proceso tradicional, integración, testing y entrega pueden concentrarse al final, acumulando incompatibilidades y defectos. Al realizar estas actividades de manera frecuente, los problemas tienden a localizarse cuando todavía son pequeños. Esto se relaciona con Lean: eliminar esperas y retrabajo, amplificar aprendizaje, entregar rápido e incorporar calidad desde el comienzo.

Las cuatro prácticas principales son **integración continua (CI), pruebas continuas, entrega continua (Continuous Delivery) y despliegue continuo (Continuous Deployment)**. Las construcciones automatizadas, la gestión de configuración, la retroalimentación y la mejora continua permiten sostenerlas.

2. Integración continua (Continuous Integration - CI)

2.1. Definición y problema que resuelve

La integración continua consiste en **incorporar frecuentemente cambios a una línea de código compartida y verificar cada integración mediante un proceso automatizado de construcción y pruebas**. Su objetivo es descubrir tempranamente conflictos, incompatibilidades y regresiones.

Problema de integración tardía: si distintos desarrolladores modifican componentes durante semanas sin combinarlos, pueden alterar interfaces, dependencias y contratos entre módulos. Integrarlo todo al final produce múltiples errores simultáneos y dificulta identificar el cambio responsable. Este fenómeno suele llamarse integración *Big Bang*.

Ejemplo: tres personas trabajan sobre productos, facturación y descuentos. Una modifica la estructura del producto; otra espera todavía el formato anterior. Con integración frecuente, la incompatibilidad se identifica en las primeras modificaciones. Con integración tardía podría descubrirse después de semanas de trabajo acumulado.

2.2. Flujo de CI

1. El desarrollador obtiene la versión reciente del código y trabaja localmente.
2. Realiza cambios pequeños y las verificaciones necesarias en su entorno privado.
3. Integra los cambios al repositorio compartido, resolviendo los conflictos que correspondan.

4. Un proceso automatizado recupera fuentes y dependencias, compila y construye el software.
5. Ejecuta pruebas unitarias, de integración u otras verificaciones configuradas.
6. Comunica los resultados. Si falla, se prioriza identificar y resolver el problema.
7. Si resulta satisfactorio, el equipo mantiene una referencia compartida sobre el estado del producto.

Kniberg, en *Scrum y XP desde las trincheras*, describe un servidor que inicia un build limpio y pruebas tras cada integración; también realiza compilaciones nocturnas y publica artefactos, documentación, informes y datos de cobertura. En algunos productos se instala automáticamente una versión en un ambiente de prueba.

2.3. Condiciones y buenas prácticas

Integrar frecuentemente. Las modificaciones pequeñas y regulares reducen la incertidumbre. Crispin y Gregory recomiendan al menos una integración diaria por integrante; los Poppendieck describen integraciones varias veces al día en determinadas prácticas.

Automatizar el build. Evitar una lista manual e irreproducible de pasos para construir el producto.

Automatizar pruebas relevantes. Compilar sin errores no demuestra que el software cumple sus requisitos.

Recibir feedback rápido. Cuanto menos tarde una verificación, más fácil resulta corregir el problema sin perder el contexto.

Corregir builds fallidos prioritariamente. Un estado compartido defectuoso puede impedir que otros integrantes trabajen confiablemente.

Mantener el código bajo control de versiones. Se necesita conocer qué fuentes, configuraciones y dependencias forman parte de cada integración.

Construir de manera reproducible. Una versión identificada debe poder reconstruirse utilizando los elementos pertinentes.

CI no es sinónimo de Git. Git es una herramienta de control de versiones; CI es una práctica de integración y verificación frecuente que utiliza repositorios y mecanismos automatizados.

3. Construcción continua y tipos de Build

3.1. Qué significa Build

Un **build** es el proceso por el que se combinan y transforman componentes fuente para generar un artefacto utilizable. Puede incluir recuperar código, resolver dependencias, compilar, generar recursos, empaquetar y verificar resultados.

Build limpio (Clean Build): comienza sin reutilizar resultados generados previamente. Proporciona una comprobación más independiente de artefactos anteriores.

Build incremental: reutiliza resultados y reconstruye solamente aquello que cambió o depende de cambios. Suele ser más rápido, pero su corrección depende de una adecuada gestión de dependencias.

3.2. Tres tipos de Build según Agile SCM

Berczuk, Appleton y Konieczka distinguen tres patrones fundamentales:

Build	Propósito	Alcance
Private System Build	Verificar los cambios antes de compartirlos.	Espacio privado del desarrollador.
Integration Build	Combinar cambios y comprobar la estabilidad compartida.	Línea de integración del equipo.
Release Build	Preparar un artefacto identificable y distribuible.	Versión candidata a entrega.

Private System Build. El desarrollador actualiza su espacio privado con los cambios recientes, resuelve conflictos y construye el sistema localmente antes de publicar. Reduce la posibilidad de introducir fallas simples en el código compartido.

Integration Build. Verifica que el conjunto de cambios del equipo sea compatible. Puede ejecutarse por commit, periódicamente o en builds nocturnos más extensos. Los resultados proporcionan una referencia del estado integrado.

Release Build. Produce una versión preparada para distribución, con fuentes y artefactos identificados, empaquetado y controles apropiados. Preparar un release **no implica necesariamente desplegarlo en producción**.

El artículo *Agile SCM - Build Management for an Agile Team* recomienda que los procedimientos de construcción y despliegue se definan tempranamente, no cuando ya se necesita entregar.

4. Pruebas continuas (Continuous Testing)

4.1. Definición

Las pruebas continuas consisten en incorporar actividades de testing durante todo el proceso de desarrollo y repetir verificaciones apropiadas para obtener información rápida sobre la calidad. No implican ejecutar absolutamente todas las pruebas con cada modificación ni automatizar todo.

Las pruebas acompañan el análisis de historias, la definición de criterios de aceptación, el diseño, la programación, la integración y la entrega. Crispin y Gregory sostienen que programación y testing deben entenderse como partes de un mismo proceso.

Beneficios: detectar defectos temprano, reducir el tiempo de diagnóstico, prevenir regresiones, evitar trabajo pendiente de pruebas y facilitar modificaciones sin perder confianza.

4.2. Automatización y testing manual

La automatización permite repetir verificaciones a bajo costo marginal cuando el número de ejecuciones es elevado. Sin embargo, el testing continuo no equivale únicamente a ejecutar scripts: incluye análisis, investigación, diseño de pruebas y juicio humano.

Las pruebas exploratorias y de usabilidad siguen siendo importantes. La automatización verifica comportamientos previstos; la exploración puede encontrar situaciones que el equipo no había imaginado.

4.3. Pirámide de automatización

La pirámide de automatización presentada por Crispin y Gregory, basada en la propuesta de Mike Cohn, plantea:

- **Base:** muchas pruebas unitarias y de componentes, rápidas y relativamente económicas.
- **Nivel intermedio:** pruebas funcionales y de aceptación a través de servicios y API, más amplias y normalmente menos rápidas.
- **Cima:** menos pruebas automatizadas mediante interfaz gráfica (GUI), normalmente más lentas y frágiles ante cambios de presentación.

El modelo no propone eliminar las pruebas de interfaz, sino reservarlas para verificar recorridos y aspectos para los cuales son apropiadas. Las pruebas exploratorias manuales complementan la pirámide.

4.4. Tipos de pruebas continuas

Pueden incorporarse pruebas unitarias, integración, regresión, aceptación, rendimiento, seguridad, confiabilidad y exploración. Su frecuencia depende del riesgo, del costo de ejecución y del tiempo de feedback que proporcionan. Una estrategia frecuente consiste en ejecutar verificaciones rápidas con cada integración y reservar pruebas costosas para ejecuciones programadas o para determinadas versiones.

4.5. TDD (Test-Driven Development)

TDD es desarrollo guiado por pruebas. Se resume como **Red - Green - Refactor**:

1. **Red:** escribir una prueba automatizada que falla porque el comportamiento todavía no existe.
2. **Green:** implementar lo mínimo necesario para que la prueba pase.
3. **Refactor:** mejorar la estructura del código sin modificar el comportamiento verificado.

Repetir el ciclo facilita cambios pequeños y diseño verificable. **TDD no equivale a todo el testing:** normalmente se concentra en unidades o componentes y debe complementarse con aceptación, integración, evaluación de atributos no funcionales y exploración.

5. Entrega continua y despliegue continuo

5.1. Continuous Delivery

La **entrega continua** busca mantener el software en condiciones de liberarse mediante integración, pruebas, builds reproducibles, empaquetado y procesos de preparación confiables. El negocio conserva la posibilidad de **decidir cuándo publicar en producción**. En la bibliografía proporcionada, la noción aparece relacionada con *Release Builds* y la posibilidad de mantener el código *ready to release*; la definición formal es una aclaración conceptual complementaria.

Por ejemplo, una nueva funcionalidad supera CI y las verificaciones previstas, se genera un artefacto identificable y se instala en pruebas. Queda preparada para publicarse cuando el negocio lo determine.

5.2. Continuous Deployment

El **despliegue continuo** automatiza además la publicación en producción de los cambios que superan los controles establecidos. No requiere una aprobación manual para cada despliegue dentro del flujo automatizado. Esto exige pruebas confiables, procedimientos controlados, observabilidad y mecanismos para responder ante problemas.

No todos los sistemas necesitan o pueden utilizar despliegue continuo; determinados entornos requieren autorizaciones, coordinación con clientes o controles regulatorios.

Aspecto	Entrega continua	Despliegue continuo
Objetivo	Software preparado para liberar	Publicación automática tras controles
Decisión de publicar	Puede ser manual	Automatizada por el proceso
Dependencias	CI, pruebas, empaquetado, control de versiones	Todo lo anterior y controles de despliegue
Alcance	Preparación confiable	Preparación más puesta en producción automática

5.3. Build, Release y Deployment

Build: construcción del software. **Release:** versión identificada y preparada para una entrega. **Deployment:** instalación o puesta en funcionamiento en un ambiente. Un build no representa necesariamente una release; una release puede existir sin estar instalada en producción; un despliegue puede hacerse en pruebas y no solamente en producción.

6. Prácticas complementarias y ejemplo integrador

6.1. Gestión de configuración

Permite controlar la identidad, el estado, las versiones, las dependencias y los cambios de los elementos del software. Es esencial para reconstruir una versión, saber qué se probó y vincular un artefacto con sus fuentes.

6.2. Retroalimentación continua

La calidad técnica no garantiza que el usuario necesite la funcionalidad. *Lean UX* propone obtener información de usuarios durante el desarrollo, validar hipótesis y observar resultados. Una función puede pasar todos los tests y ser confusa o innecesaria.

6.3. Refactorización y mejora continua

La refactorización mejora la estructura del código preservando el comportamiento externo. Las pruebas de regresión proporcionan una red de seguridad. Las retrospectivas permiten revisar el proceso y disminuir dificultades recurrentes. La deuda técnica debe gestionarse para no dificultar las entregas futuras.

6.4. Ejemplo de extremo a extremo

Un equipo agrega cupones de descuento a un comercio electrónico:

1. Define criterios de aceptación sobre cupones válidos, vencidos y compras elegibles.
2. Desarrolla la regla, por ejemplo con TDD, y ejecuta pruebas locales.
3. Integra el cambio al repositorio; CI realiza build y pruebas automáticas.
4. Ejecuta verificaciones funcionales, de integración y exploratorias según el riesgo.
5. Genera un Release Build con una versión identificada y preparada.

6. En entrega continua, el negocio decide cuándo publicarla; en despliegue continuo, una automatización puede realizarlo tras superar los controles.
7. El equipo observa el uso real, recibe feedback y mejora el producto.

6.5. Beneficios, dificultades y métricas

Beneficios: defectos descubiertos temprano, menos retrabajo, builds reproducibles, versiones más frecuentes, feedback rápido y menor esfuerzo manual repetitivo.

Dificultades: inversión en automatización, mantenimiento de ambientes y pruebas, dependencias complejas, presión para corregir builds fallidos, cultura de colaboración y riesgo de falsa confianza si las pruebas son insuficientes.

Indicadores: frecuencia de integración, duración de build, tasa de builds fallidos, tiempo de recuperación, defectos posteriores a la entrega, frecuencia de release, tiempo desde cambio hasta entrega y fiabilidad de las pruebas.

7. Prácticas continuas: desarrollo detallado, criterios y problemas habituales

7.1. Por qué «continuo» no quiere decir «todo, todo el tiempo»

Las prácticas continuas buscan disminuir el tiempo que pasa entre producir un cambio y recibir información útil. La palabra «continuo» expresa frecuencia e integración al proceso, no la obligación de ejecutar absolutamente todas las actividades después de cada línea de código. Un cambio pequeño puede pasar por pruebas unitarias y de integración rápidas; una batería pesada de rendimiento puede ejecutarse en momentos previamente definidos, siempre que el riesgo esté adecuadamente tratado.

Cuando se administra software como **producto** y no como una entrega que finaliza en una fecha, importan su evolución, operación, aprendizaje del uso real, mantenimiento y capacidad para responder a necesidades nuevas. La integración frecuente, los builds reproducibles y las pruebas automatizadas reducen el costo de introducir cambios y facilitan entregar incrementos pequeños.

Un escenario opuesto es la **integración tardía o «big bang»**: varios desarrolladores trabajan mucho tiempo en componentes separados y únicamente al final intentan combinar las modificaciones. Si existen diferencias incompatibles, cambios de interfaz o suposiciones contradictorias, resulta difícil identificar cuándo y dónde se introdujo el problema.

7.2. Integración continua: secuencia completa

Una secuencia didáctica de **Continuous Integration (CI)** comprende: (1) obtener del repositorio compartido la versión más reciente; (2) incorporar y probar un cambio pequeño en el entorno de desarrollo; (3) verificar localmente con un *private build* y las pruebas apropiadas; (4) compartir el cambio según la política de ramas e integración; (5) iniciar un *integration build* automatizado; (6) compilar, resolver dependencias, ejecutar pruebas y registrar resultados; (7) dar feedback inmediato; y (8) corregir una falla de integración prioritariamente antes de acumular más cambios.

La CI requiere **frecuencia, integración real y verificación automatizada**. Usar Git o tener un repositorio no significa practicar CI si se integran grandes cambios solamente cada pocas semanas. Tampoco alcanza con compilar una vez por día si las modificaciones incompatibles permanecen demasiado tiempo aisladas.

El libro *Implementing Lean Software Development* de Poppendieck presenta la integración continua como un mecanismo para disminuir el riesgo asociado a separar durante mucho tiempo el trabajo de distintos

desarrolladores. Kniberg, en *Scrum y XP desde las trincheras*, describe un entorno con builds y pruebas automatizadas mediante Maven y QuickBuild y distingue verificaciones por cambio y builds limpios nocturnos.

Falla de compilación: el sistema no pudo construirse. **Falla de pruebas:** se construyó pero un comportamiento verificado no coincidió con el esperado. **Falla de ambiente o infraestructura:** el resultado puede no representar necesariamente un defecto del código de negocio. Las tres situaciones requieren diagnóstico; no se deben ignorar builds fallidos porque son frecuentes.

7.3. Build, pruebas y despliegue: conceptos separados

Build: proceso de transformación de fuentes y dependencias en artefactos, que puede incluir compilación, empaquetado, generación y tareas auxiliares. **Build limpio:** parte de un estado sin resultados previos que puedan ocultar problemas. **Build incremental:** intenta reutilizar resultados existentes y reconstruir solamente lo afectado. El primero ayuda a comprobar reproducibilidad; el segundo puede acelerar el trabajo cotidiano.

Private System Build: comprueba la combinación de cambios del desarrollador con el estado actual de su entorno antes de compartirlos. **Integration Build:** verifica el sistema compartido resultante de incorporar cambios de distintos desarrolladores. **Release Build:** genera una versión identificada, con artefactos adecuados para entregar o distribuir. Estos tres patrones son desarrollados por Berczuk, Appleton y Konieczka en *Agile SCM - Build Management for an Agile Team*.

Es incorrecto afirmar que todo *build* despliega en producción. Una construcción puede producir una biblioteca, un ejecutable, un paquete o una imagen instalable. Una **release** identifica una versión preparada para distribución y un **deployment** la instala o pone en funcionamiento en un ambiente, que puede ser desarrollo, pruebas, preproducción o producción.

Concepto	Pregunta que responde	Resultado típico
Private Build	¿Mi cambio local está en condiciones de compartirse?	Construcción y pruebas locales
Integration Build	¿Los cambios compartidos funcionan juntos?	Build y pruebas de integración del equipo
Release Build	¿Existe una versión identificable y distribuible?	Paquete o artefacto versionado
Continuous Delivery	¿Podemos liberar confiablemente cuando el negocio lo decida?	Versión preparada para entrega
Continuous Deployment	¿Puede publicarse automáticamente tras controles?	Publicación automatizada bajo políticas

7.4. Una estrategia de testing continuo que no dependa de una sola herramienta

Las verificaciones deben distribuirse según **velocidad, riesgo, costo y utilidad del feedback**. Una estrategia frecuente puede reservar para cada commit: análisis rápido, compilación y pruebas unitarias; después de integrar cambios: pruebas de integración y funcionales apropiadas; periódicamente o en hitos definidos: regresiones extensas, rendimiento, seguridad y otras verificaciones más costosas. El esquema exacto depende del sistema.

La **pirámide de pruebas** ayuda a evitar una suite compuesta mayoritariamente por pruebas GUI lentas y frágiles. La base contiene numerosas pruebas unitarias; el medio contiene pruebas funcionales, de componentes o servicios; la cima reserva pruebas de interfaz para recorridos importantes. Ninguna

proporción numérica es obligatoria. Las pruebas exploratorias manuales se mantienen porque permiten investigar comportamientos no previstos en scripts.

Prueba automatizada no es sinónimo de **prueba suficiente**. Un conjunto de miles de tests que solo verifica casos felices puede omitir valores límite, errores de integración o riesgos relevantes. Deben revisarse cobertura, calidad de aserciones, independencia de datos y confiabilidad del entorno.

7.5. TDD, ATDD y testing continuo: comparación

TDD (desarrollo guiado por pruebas) utiliza el ciclo «Red - Green - Refactor»: escribir una prueba que falla, implementar lo necesario para que pase y mejorar el diseño sin alterar el comportamiento. Normalmente ayuda a diseñar unidades o componentes y facilita mantener código verificable. **ATDD** define ejemplos y pruebas de aceptación con participación de negocio, desarrollo y testing antes de implementar la funcionalidad. **Testing continuo** es más amplio: comprende actividades de verificación y evaluación durante toda la evolución del producto. Una organización puede aplicar pruebas continuas sin utilizar TDD en cada línea, aunque TDD suele apoyarla.

Ejemplo de ATDD: «un cupón del 15 % debe aplicarse solamente si está vigente, corresponde al producto y el cliente cumple sus condiciones». El equipo define una tabla de ejemplos, acuerda los resultados y usa esas expectativas para orientar la implementación. TDD puede utilizarse después para construir las funciones de validación de cupones con pruebas unitarias.

7.6. Entrega continua versus despliegue continuo: advertencia bibliográfica

La distinción formal entre **Continuous Delivery** y **Continuous Deployment** que usamos en esta guía es una **aclaración técnica complementaria**: la bibliografía proporcionada desarrolla builds, automatización e integración, pero no formula extensamente ambas definiciones modernas. En **Continuous Delivery** la versión se mantiene liberable, pero la puesta en producción puede quedar bajo decisión humana. En **Continuous Deployment** la publicación se ejecuta automáticamente cuando se satisfacen los controles establecidos. Ninguno de los dos elimina la necesidad de gestionar seguridad, operaciones y riesgo de negocio.

Ejemplo: un banco podría generar artefactos perfectamente preparados y verificados cada día, pero publicar solo después de una aprobación de cambio. Estaría utilizando prácticas compatibles con entrega continua sin publicar automáticamente cada commit.

7.7. SCM, trazabilidad y reproducibilidad

La **gestión de configuración de software (SCM)** mantiene la identificación y control de fuentes, componentes, dependencias, documentación y versiones relevantes. Permite saber qué código corresponde a un artefacto, qué pruebas se ejecutaron sobre él y cómo reconstruirlo. Un repositorio sin políticas de versiones, dependencias o releases puede ser insuficiente para garantizar integridad y trazabilidad.

Si aparece una falla en producción, es importante responder: «¿Qué versión se encuentra instalada?», «¿Qué cambios contiene?», «¿Qué pruebas se ejecutaron?», «¿Podemos reconstruir ese artefacto?» y «¿Cómo volvemos a una versión anterior si es necesario?». Estas preguntas relacionan CI/CD con los temas de administración de configuración y aseguramiento de calidad de la Unidad 4.

7.8. Ejemplo de extremo a extremo y fallas posibles

Un equipo desarrolla una función para recuperar contraseñas. (1) El negocio acuerda reglas de expiración, mensajes y restricciones. (2) El equipo escribe pruebas unitarias para la generación y validación de tokens. (3) Una prueba de aceptación describe el flujo completo. (4) Se realiza un private build. (5) El código se integra al repositorio y se ejecutan verificaciones automáticas. (6) El equipo realiza pruebas exploratorias y evalúa

riesgos de seguridad. (7) Se prepara una release identificada. (8) El negocio decide si autoriza la publicación o el pipeline la publica bajo una política automatizada. (9) El equipo observa incidentes y feedback de uso.

Problema posible 1: un test unitario pasa, pero el servicio de correo está mal configurado. Lo detectan las pruebas de integración o el monitoreo del ambiente. **Problema posible 2:** el sistema cumple los requisitos funcionales pero el enlace de recuperación confunde a los usuarios. Lo detectan las pruebas de usabilidad o la retroalimentación real. **Problema posible 3:** el pipeline falla aleatoriamente por datos compartidos. Debe mejorarse el aislamiento de las pruebas. **Problema posible 4:** hay una vulnerabilidad que no aparece en la suite funcional. Debe ampliarse el tratamiento de riesgos de seguridad.

7.9. Métricas y señales de alarma

Un equipo puede analizar duración del build, frecuencia de integración, porcentaje de construcciones que fallan, tiempo para reparar una construcción rota, defectos después de la entrega, tiempo desde el commit hasta una release, cantidad de cambios incluidos por entrega y estabilidad de las pruebas. Ninguna métrica aislada mide toda la calidad del proceso.

Señales de alarma: integraciones que solo ocurren al final de la iteración, builds rojos durante largos períodos, pruebas de aceptación que se ejecutan después de dar por finalizadas las historias, versiones sin identificación reproducible, scripts que fallan por razones no determinadas y falta de pruebas exploratorias.

7.10. Preguntas típicas del parcial

¿Un repositorio compartido es suficiente para CI? No. Hace falta integración frecuente y verificación automática útil. **¿Se deben ejecutar todas las pruebas con cada commit?** No necesariamente; se seleccionan verificaciones según costo y necesidad de feedback. **¿Un Release Build es una puesta en producción?** No; prepara un artefacto liberable. **¿TDD reemplaza las pruebas de sistema?** No; las complementa. **¿Pruebas continuas significan eliminar testing manual?** No; exploración y evaluación de uso siguen siendo necesarias. **¿Por qué SCM es importante?** Porque permite identificar qué se construyó, probó y liberó, y mantiene la integridad de los cambios.

Resumen para el parcial · Unidad 3

CI: integrar frecuentemente cambios en un código compartido y verificarlos automáticamente. Git solo no equivale a CI. **Tres builds de Agile SCM:** Private System Build, Integration Build y Release Build. **Pruebas continuas:** acompañar todo el desarrollo y obtener feedback temprano; automatización no sustituye exploración ni criterio humano. **TDD:** Red - Green - Refactor. **Continuous Delivery:** software listo para liberar, con decisión de publicación potencialmente manual. **Continuous Deployment:** publicación automática tras los controles. **Build, release y deployment son conceptos diferentes.** Gestión de configuración, retroalimentación, refactorización y mejora continua hacen sostenible el enfoque.

Bibliografía: Berczuk, Appleton y Konieczka, *Agile SCM - Build Management for an Agile Team*; Poppendieck y Poppendieck, *Implementing Lean Software Development*, capítulo 8; Kniberg, *Scrum y XP desde las trincheras*; Crispin y Gregory, *Agile Testing*, capítulos 14 y 21; Gothelf y Seiden, *Lean UX*.

UNIDAD 4 · ASEGURAMIENTO DE CALIDAD DE PROCESO Y PRODUCTO

1. Conceptos generales sobre calidad

1.1. Definición de calidad

La cátedra define la **calidad** como el conjunto de aspectos y características de un producto o servicio relacionados con su capacidad de satisfacer **necesidades manifiestas o implícitas**. Las explícitas están declaradas o acordadas; las implícitas son expectativas razonables que quizá no aparezcan escritas, como la protección de datos o un tiempo de respuesta adecuado.

Un sistema puede implementar todas las funciones detalladas en su especificación y aun así presentar baja calidad: por ejemplo, si sus tiempos de respuesta impiden utilizarlo o si expone datos confidenciales. Cumplir una lista de requisitos es relevante, pero no siempre suficiente para satisfacer las necesidades reales.

1.2. Calidad relativa al usuario, contexto y tiempo

La calidad es relativa: depende de qué persona la evalúa, qué necesita, cuál es el entorno y en qué momento se utiliza el producto. Federico Toledo, en *Introducción a las Pruebas de Sistemas de Información*, recupera la idea de Jerry Weinberg de que la calidad es valor para una persona; Cem Kaner la amplía considerando a la persona interesada.

Un sistema hospitalario puede ser valorado por seguridad y disponibilidad; una aplicación de entretenimiento, por facilidad de uso; un equipo técnico, por mantenibilidad. No existe una jerarquía universal que ordene estos atributos sin contexto.

Falacia del Nirvana: comparar soluciones reales con una alternativa perfecta e inexistente y concluir que las primeras carecen de valor. Reconocer que la perfección es inalcanzable no equivale a aceptar riesgos arbitrarios: significa establecer objetivos de calidad realistas y apropiados.

1.3. Cinco visiones de la calidad

La diapositiva 9 de PPQA distingue cinco perspectivas:

Visión	Significado	Ejemplo
Trascendental	Excelencia que se reconoce, aunque no sea fácil cuantificar.	Experiencia global considerada sobresaliente.
Del producto	Características inherentes y medibles.	Fiabilidad, rendimiento y mantenibilidad.
Del usuario	Adecuación para satisfacer necesidades reales.	Operaciones comprensibles y útiles para una persona.
De manufactura	Conformidad con especificaciones y procesos establecidos.	Cumplimiento de reglas documentadas.
Basada en el valor	Beneficio obtenido en relación con el costo.	Una solución más sencilla que satisface el negocio a menor costo.

Las visiones no se excluyen. Una evaluación útil debe considerar características técnicas, conformidad, expectativas del usuario y restricciones económicas.

1.4. Calidad programada, realizada y necesaria

La presentación de la cátedra propone tres dimensiones que deberían coincidir:

- **Calidad programada:** nivel y características de calidad planificados.
- **Calidad realizada:** lo que efectivamente se construyó.
- **Calidad necesaria:** lo que requieren realmente cliente y usuario.

Si el equipo programa y construye una función que nadie necesita, existe desperdicio. Si una necesidad no fue identificada, el producto puede causar insatisfacción aunque se cumpla todo el plan. Si la necesidad fue identificada pero la implementación es defectuosa, falla la calidad realizada. La gestión de calidad busca reducir estas diferencias.

1.5. Diez principios de calidad de la cátedra

La diapositiva 8 de PPQA resume diez ideas:

1. La calidad no se inyecta ni se compra: debe estar incorporada durante todo el desarrollo.
2. Es un esfuerzo de todos.
3. Las personas son fundamentales; necesitan capacitación.
4. Se requiere apoyo gerencial, aunque las mejoras pueden comenzar localmente.
5. Se debe liderar con el ejemplo.
6. No se puede controlar adecuadamente lo que no se mide.
7. Conviene mantener la simplicidad y comenzar por lo básico.
8. El aseguramiento de calidad debe planificarse.
9. Aumentar la cantidad de pruebas no aumenta automáticamente la calidad.
10. El esfuerzo de calidad debe ser razonable para el negocio.

Consecuencia: una organización no debería intentar agregar la calidad exclusivamente en las pruebas finales. Los requisitos, el diseño, la programación, el proceso y la evaluación contribuyen a la calidad del producto.

2. Calidad de producto y calidad de proceso

2.1. Calidad del producto

La **calidad de producto** analiza las características del software obtenido y su capacidad de satisfacer los requisitos y necesidades. Comprende atributos funcionales y no funcionales.

La cátedra presenta el modelo **ISO/IEC 25010:2011**, que distingue ocho características principales:

Característica	Significado
Adecuación funcional	Las funciones cubren correcta y completamente las necesidades.
Eficiencia de desempeño	Tiempos de respuesta, recursos y capacidad.

Característica	Significado
Compatibilidad	Coexistencia e interoperabilidad con otros sistemas.
Usabilidad	Facilidad para comprender, aprender y utilizar el sistema.
Fiabilidad	Continuidad de funcionamiento, tolerancia a fallas y recuperación.
Seguridad	Confidencialidad, integridad, autenticación y otras protecciones.
Mantenibilidad	Analizabilidad, modularidad, modificación, reuso y prueba.
Portabilidad	Adaptación, instalación y reemplazo en otros entornos.

La familia ISO/IEC 25000 también contempla calidad en uso, relacionada con los resultados obtenidos por usuarios en contextos reales, y calidad de datos (ISO/IEC 25012:2008). La clasificación anterior corresponde a la versión del modelo estudiada en la cátedra, no a la actualización de versiones posteriores.

Calidad interna y externa: Kniberg distingue entre características internas, como buen diseño y facilidad de cambio, y externas, perceptibles en el uso. Un producto puede funcionar hoy y tener una arquitectura muy difícil de mantener. Descuidar la calidad interna puede comprometer la evolución futura.

2.2. Calidad de proceso

La **calidad del proceso** analiza cómo se organizan, ejecutan, evalúan y mejoran las actividades utilizadas para desarrollar y mantener software. El proceso comprende métodos, actividades, herramientas, responsabilidades, procedimientos y criterios de control.

La cátedra explica que el proceso representa un factor que la organización puede definir y mejorar sistemáticamente. Esto no significa que las personas y las condiciones del negocio carezcan de importancia, sino que contar con un proceso explícito facilita prevenir problemas recurrentes.

Ejemplo: si los requisitos suelen llegar ambiguos a desarrollo, corregir cada defecto de código no resolverá la causa. Mejorar las prácticas de elicitación, revisión y aprobación de requisitos puede prevenir defectos futuros.

La presentación plantea un ciclo de **definir procesos, desarrollar el producto, evaluar la calidad y mejorar o estandarizar procesos según los resultados**. Insiste además en que no se deben utilizar prácticas inadecuadas solamente porque figuren en un estándar.

Calidad de proceso	Calidad de producto
Evalúa cómo se desarrolla.	Evalúa qué se construyó.
Se apoya en estándares, seguimiento y auditorías.	Se apoya en pruebas, revisiones y mediciones de atributos.
Busca capacidad de repetición y prevención.	Busca conformidad y satisfacción de necesidades.
No garantiza automáticamente un producto adecuado.	No demuestra automáticamente que el proceso sea bueno.

Ambas dimensiones son complementarias: procesos sólidos aumentan las posibilidades de construir productos de calidad, pero no sustituyen la validación de las necesidades reales.

3. Importancia de trabajar para y con calidad

3.1. Importancia

El software participa en procesos de negocio y actividades cotidianas. Un defecto puede generar demoras, información incorrecta, pérdidas económicas, problemas de privacidad o daños graves cuando el sistema es crítico. La calidad debe gestionarse atendiendo al **riesgo**, es decir, a la probabilidad de falla y a sus posibles consecuencias.

La cátedra enumera problemas frecuentes en proyectos: atrasos, costos excedidos, compromisos incumplidos, requisitos poco claros, software que no hace lo esperado, jornadas excesivas y dificultades para localizar componentes. El aseguramiento permite reconocer y tratar causas relacionadas con la forma de trabajo.

Detección temprana: un problema de requisitos descubierto durante el análisis suele evitar más retrabajo que si se detecta después de construir y desplegar el producto. Los materiales ilustran esta tendencia con el crecimiento del costo de corrección a lo largo del ciclo de vida; los multiplicadores de esas figuras no deben asumirse como universales.

Responsabilidad compartida: analistas, desarrolladores, testers, responsables de proyecto, usuarios y gerencia contribuyen a la calidad desde sus respectivos roles. El equipo de QA no puede compensar por sí solo las decisiones deficientes tomadas en otras actividades.

3.2. Ventajas

Menos defectos y retrabajo. Revisar requisitos y diseños permite detectar causas de errores antes de que se propaguen.

Menor costo de fallas. La prevención puede reducir correcciones, soporte, interrupciones y daños reputacionales.

Mayor satisfacción del cliente. El producto responde mejor a expectativas explícitas e implícitas.

Previsibilidad. Los procesos definidos y evaluados facilitan conocer el avance y detectar desviaciones.

Mantenibilidad. El diseño y código de calidad permiten evolucionar el producto con menos riesgo.

Menor exposición a riesgos. Los controles pueden reducir la probabilidad o impacto de fallas críticas.

Aprendizaje organizacional. Las evaluaciones y retrospectivas permiten modificar procesos y evitar defectos repetidos.

3.3. Costos, limitaciones y dificultades

Inversión inicial: formación, herramientas, plantillas, ambientes, actividades de revisión y automatización.

Tiempo de coordinación: planificar controles, reunir participantes, registrar resultados y hacer seguimiento consume esfuerzo.

Resistencia cultural: algunas personas pueden interpretar las auditorías o revisiones como críticas personales.

Riesgo de burocracia: exigencias documentales o aprobaciones que no aportan valor pueden ralentizar el flujo.

Dificultad de medición: no existe una única métrica que represente las necesidades de todos los interesados.

Falsa sensación de seguridad: muchas pruebas aprobadas o un proceso muy documentado no garantizan ausencia de defectos ni satisfacción del usuario.

Equilibrio costo-riesgo: los controles deben ser proporcionales al negocio; el testing exhaustivo de todas las combinaciones es imposible en sistemas reales.

La calidad no es una desventaja en sí. Las desventajas aparecen cuando su implementación requiere recursos o se convierte en una actividad desproporcionada o inadecuada.

4. QA, planificación, QC, testing, verificación y validación

4.1. Administración de calidad

La presentación distingue tres actividades:

Aseguramiento de Calidad (QA/SQA): establece estándares y procedimientos organizacionales y evalúa las condiciones necesarias para alcanzar calidad. Tiene una orientación principalmente preventiva.

Planificación de la Calidad: selecciona y adapta los estándares aplicables al proyecto y define objetivos, atributos, métodos y criterios de evaluación.

Control de Calidad (QC): comprueba el cumplimiento de estándares y procedimientos y evalúa productos o procesos mediante revisiones, evaluaciones automáticas y mediciones.

Testing: conjunto de actividades estáticas y dinámicas que proporciona información sobre el software y detecta defectos. El testing contribuye a la calidad, pero no la garantiza por sí solo.

Concepto	Pregunta práctica
QA	¿Están establecidos y evaluados los procesos que deberían permitir calidad?
Planificación	¿Qué calidad buscamos y cómo la evaluaremos?
QC	¿Se cumplen los criterios, procedimientos y resultados exigidos?
Testing	¿Qué información obtenemos al examinar o ejecutar el software?

4.2. Verificación y validación

Verificación: «¿Estamos construyendo correctamente el sistema?». Contrasta productos de trabajo y su realización con especificaciones y criterios aplicables.

Validación: «¿Estamos construyendo el sistema correcto?». Analiza si el producto satisface el propósito previsto y las necesidades reales.

Un formulario que reproduce perfectamente un requisito que exige quince datos puede estar verificado y, sin embargo, ser inadecuado si los usuarios abandonan el proceso porque se solicitan datos innecesarios. Por eso, ambas actividades son necesarias.

5. Actividades relacionadas con SQA

5.1. Objetivos de SQA

La presentación sobre Auditorías de Software establece tres objetivos: realizar controles apropiados del software y el proceso; asegurar el cumplimiento de estándares y procedimientos; e informar a gerencia los defectos del producto, proceso o estándares para permitir su resolución.

La cátedra indica que las funciones de SQA comprenden desarrollar técnicas, métodos, herramientas y estándares; evaluar la planificación del proyecto; evaluar requisitos, diseño y programación; revisar el proceso de integración y pruebas; evaluar planificación y control; y adaptar las actividades de aseguramiento a cada proyecto.

Ejemplos: comprobar que el Plan de Proyecto contemple pruebas y configuración; verificar que las modificaciones a requisitos sigan un procedimiento; revisar que se realicen inspecciones de código previstas; evaluar el registro y cierre de defectos; comprobar que el proceso de pruebas utiliza versiones identificadas.

5.2. Grupo de Aseguramiento de Calidad (GAC)

El GAC coordina o realiza actividades de evaluación y seguimiento de calidad. Debe conservar suficiente **independencia** respecto de los proyectos que evalúa. La cátedra recomienda que no dependa directamente del gerente del proyecto, que exista poca distancia jerárquica hasta la gerencia superior y que reporte a responsables comprometidos con la calidad.

La independencia contribuye a evitar presión para ocultar problemas, sin impedir la colaboración con los equipos. El grupo debe basar sus conclusiones en criterios y evidencias.

5.3. Estándares de proceso y producto

Estándares de producto establecen características que deben cumplir componentes o artefactos, como formatos de documentación o convenciones de programación. **Estándares de proceso** determinan cómo deben realizarse actividades, como revisión de requisitos o control de configuración. Pueden ser internacionales, nacionales, organizacionales o específicos de proyecto.

5.4. Revisiones y auditorías

Una revisión técnica examina el contenido de un producto de trabajo en busca de defectos. Una auditoría evalúa objetivamente el cumplimiento de criterios, procedimientos y estándares. Puede auditarse si se realizaron las revisiones exigidas sin que el auditor repita necesariamente toda la revisión técnica.

6. Principales modelos de calidad: CMMI, SPICE e ISO

6.1. Propósitos diferentes

Los modelos y estándares pueden servir para **definir procesos, evaluarlos, mejorarlos o describir características de calidad del producto**. No todos resuelven el mismo problema. CMMI estudia madurez y capacidad de procesos; SPICE se orienta a evaluar capacidad de procesos; ISO es una organización de normalización con familias normativas de alcances diversos.

6.2. CMMI: concepto, origen y constelaciones

CMMI (Capability Maturity Model Integration) es un marco de prácticas para evaluar y mejorar la madurez y capacidad de los procesos organizacionales. Surge de la evolución e integración de modelos del ámbito del Software Engineering Institute. La cátedra utiliza principalmente **CMMI-DEV versión 1.3**.

Las constelaciones clásicas estudiadas son **CMMI-DEV** (desarrollo), **CMMI-SVC** (servicios) y **CMMI-ACQ** (adquisiciones). Las diapositivas desarrollan especialmente CMMI-DEV.

Madurez: grado de consolidación de un conjunto de procesos de la organización. **Capacidad:** desempeño y gestión de un proceso o área específica. En CMMI v1.3 se estudian dos representaciones: por etapas (*staged*) para madurez organizacional, y continua (*continuous*) para capacidad por área de proceso.

6.3. Cinco niveles de madurez CMMI

Nivel	Nombre	Características
1	Inicial	Procesos impredecibles, reactivos o improvisados.
2	Administrado	Los proyectos planifican, controlan y registran el trabajo.
3	Definido	Procesos organizacionales establecidos y adaptados por proyectos.
4	Administrado cuantitativamente	Objetivos cuantitativos y control del desempeño mediante mediciones.
5	Optimizado	Mejora continua, análisis de causas y prevención sistemática.

Nivel 1: el éxito depende demasiado de esfuerzos individuales y es difícil repetirlo. **Nivel 2:** se establecen mecanismos básicos de gestión de proyectos, requisitos, calidad, medición y configuración. **Nivel 3:** los proyectos utilizan procesos estándares organizacionales que pueden adaptarse. **Nivel 4:** se controla el comportamiento de procesos mediante técnicas cuantitativas. **Nivel 5:** se utilizan resultados y análisis causal para mejoras sistemáticas.

6.4. Representaciones y áreas CMMI-DEV v1.3

La **representación por etapas** utiliza cinco niveles de madurez del 1 al 5. La **continua**, según la versión estudiada, contempla niveles de capacidad de 0 a 5 para áreas de proceso seleccionadas. No deben mezclarse ambas clasificaciones.

Nivel 2 (siete áreas): Administración de Requisitos (REQM); Planificación de Proyectos (PP); Monitoreo y Control de Proyectos (PMC); Administración de Acuerdos con Proveedores (SAM); Medición y Análisis (MA); Aseguramiento de Calidad de Proceso y Producto (PPQA); Administración de Configuración (CM).

Nivel 3 (once áreas): Desarrollo de Requisitos (RD); Solución Técnica (TS); Integración del Producto (PI); Verificación (VER); Validación (VAL); Foco en el Proceso Organizacional (OPF); Definición del Proceso Organizacional (OPD); Capacitación Organizacional (OT); Administración Integrada de Proyectos (IPM); Administración de Riesgos (RSKM); Análisis y Resolución de Decisiones (DAR).

Nivel 4 (dos áreas): Desempeño del Proceso Organizacional (OPP); Administración Cuantitativa de Proyectos (QPM).

Nivel 5 (dos áreas): Análisis Causal y Resolución (CAR); Administración del Desempeño Organizacional (OPM).

Dato fundamental para el parcial: PPQA pertenece al nivel 2 y a la categoría de Soporte. Una evaluación formal de CMMI y el uso de sus prácticas para mejorar no son la misma actividad. No debe confundirse la evaluación de madurez con una certificación ISO de sistema de gestión.

6.5. CMMI y Agile

CMMI se enfoca en prácticas de proceso y mejora organizacional; Agile enfatiza colaboración, entrega incremental y respuesta al cambio. No son necesariamente incompatibles: un equipo puede desarrollar iterativamente y al mismo tiempo controlar configuración, gestionar requisitos, medir y evaluar objetivamente sus procesos. El conflicto aparece cuando se traducen los modelos a burocracia sin valor.

6.6. SPICE

SPICE es un enfoque histórico de evaluación y mejora de capacidad de procesos, relacionado con **ISO/IEC 15504**. La presentación de la cátedra lo menciona junto con los modelos de mejora, pero **no desarrolla detalladamente sus niveles**; esta explicación corresponde al modelo clásico, como complemento técnico.

Nivel	Nombre	Significado
0	Incompleto	No se alcanza el propósito del proceso.
1	Realizado	El proceso consigue su propósito.
2	Gestionado	Se planifica, supervisa y controla el desempeño y los productos de trabajo.
3	Establecido	Se define y despliega un proceso estándar.
4	Predecible	Se utiliza medición y control cuantitativo.
5	En optimización	Se mejora sistemáticamente el desempeño.

Los atributos clásicos evalúan desempeño del proceso (nivel 1), gestión del desempeño y del producto de trabajo (2), definición y despliegue (3), medición y control (4), innovación y optimización (5). SPICE se centra en determinar capacidad por proceso y orientar decisiones de mejora.

6.7. Normas ISO relevantes

ISO 9001: requisitos para un sistema de gestión de calidad de la organización. No certifica que cada producto esté libre de defectos.

ISO/IEC 90003: guía de aplicación de ISO 9001 al software.

ISO/IEC 12207: marco de procesos del ciclo de vida del software, desde definición hasta mantenimiento y otras actividades.

ISO/IEC 25000 (SQuaRE): familia de calidad de sistemas y software, requisitos y evaluación.

ISO/IEC 25010:2011: modelo de calidad del producto y calidad en uso; la cátedra presenta ocho características de producto.

ISO/IEC 25012:2008: modelo de calidad de datos.

Las normas 9001, 90003 y 12207 se incluyen como contexto complementario. La presentación de la cátedra desarrolla explícitamente la familia 25000 y el modelo 25010:2011.

6.8. Comparación de modelos

Referencia	Enfoque	Pregunta
CMMI	Madurez y capacidad de procesos	¿Qué tan consolidados están los procesos organizacionales?
SPICE	Capacidad por proceso	¿Qué capacidad presenta cada proceso evaluado?
ISO 9001	Sistema de gestión de calidad	¿La organización gestiona calidad conforme a los requisitos del sistema?
ISO/IEC 12207	Procesos del ciclo de vida	¿Qué procesos forman parte de un ciclo de vida de software?
ISO/IEC 25010	Características de calidad	¿Qué atributos deben evaluarse en el producto y en su uso?

6.9. Modelo IDEAL

La cátedra presenta IDEAL como enfoque de mejora de procesos:

1. **Initiating:** iniciar, comprender el contexto y obtener apoyo.
2. **Diagnosing:** diagnosticar los procesos y problemas actuales.
3. **Establishing:** establecer objetivos y planes de mejora.
4. **Acting:** actuar, implementar y evaluar cambios o pilotos.
5. **Learning:** aprender de los resultados y ajustar el enfoque.

IDEAL **no es una escala de madurez**; es una manera de organizar la mejora.

---

7. Auditorías de software

7.1. Concepto, objetivos y razones

Una **auditoría de software** es una evaluación independiente y objetiva de productos o procesos para determinar su conformidad con estándares, lineamientos, especificaciones y procedimientos. La presentación cita IEEE Std 1028-1988: los criterios deben establecer la forma o contenido esperado de productos, los procesos por los cuales se construyen y cómo se determinará su cumplimiento.

Se audita para obtener una opinión objetiva, detectar potenciales fuentes de insatisfacción del cliente, comprobar compromisos y reconocer oportunidades de mejora. La auditoría no es una búsqueda de culpables: se evalúan criterios y evidencias, y se reportan desviaciones para su corrección.

7.2. Tipos de auditorías de software

Auditoría de Proyecto. Comprueba el cumplimiento del proceso de desarrollo y la consistencia entre productos de trabajo. Puede verificar si las interfaces son coherentes con la ERS, si los requisitos están considerados en el Plan de Verificación y Validación, si el diseño satisface la ERS y si el código corresponde al DDS.

Auditoría de Configuración Funcional. Evalúa si el software construido corresponde con los requisitos funcionales especificados y si se dispone de evidencia de su cumplimiento. La matriz de rastreabilidad permite relacionar requisitos y otros productos.

Auditoría de Configuración Física. Comprueba que el software entregado y su documentación técnica sean consistentes. Por ejemplo, identifica discrepancias entre el código construido y el documento de arquitectura de la versión.

Las auditorías de proyecto deben estar previstas en el **PACS (Plan de Aseguramiento de Calidad de Software)**, que identifica responsables y actividades apropiadas.

Tipo	Pregunta principal
Proyecto	¿Se están cumpliendo procesos y criterios de desarrollo y existe consistencia entre entregables?
Configuración funcional	¿El producto implementa lo especificado funcionalmente?
Configuración física	¿El producto real coincide con su configuración y documentación?

7.3. Roles

Gerente de SQA: prepara el plan de auditorías, calcula costos, asigna recursos y gestiona la resolución de no conformidades.

Auditor: coordina alcance y fecha, recopila y analiza evidencias, realiza entrevistas, elabora informes y verifica acciones correctivas.

Auditado: colabora, entrega documentación, responde las observaciones, prepara acciones y comunica su cumplimiento.

7.4. Proceso de auditoría: cuatro fases

Fase 1. Preparación y planificación. Auditor y auditado acuerdan fecha, lugar, objetivo, alcance y participantes. Se solicitan y preparan documentos preliminares. El auditor establece los criterios contra los cuales evaluará. Resultado: una evaluación organizada y conocida por los participantes.

Fase 2. Ejecución. Comienza con una **reunión de apertura** para presentar a los participantes, explicar alcance, formalismos y proceso. Después se realizan entrevistas, observación, muestreo, revisión de registros y comprobación mediante checklist. El auditor escucha, registra y pregunta; debe confirmar que comprendió las respuestas. Es recomendable comenzar con preguntas abiertas (cómo, quién, cuándo, qué) y utilizar preguntas cerradas para precisar detalles.

Fase 3. Análisis y reporte. El auditor examina el checklist, las notas y documentos, compara las evidencias con los criterios y prepara resultados preliminares. En la **reunión de cierre** comunica hallazgos y aclara interpretaciones; luego emite el **informe final** con los resultados y solicitudes de acción.

Fase 4. Seguimiento. El auditado analiza las desviaciones y elabora un plan de acción con responsables y fechas. El auditor revisa el plan, comprueba las correcciones y cierra los hallazgos resueltos. Informar que una acción se realizó no equivale a demostrar que solucionó el problema.

7.5. Checklist e informe

El **checklist** contiene fecha, auditor, auditados y roles, proyecto, fase, objetivos, alcance y preguntas. Ejemplos para planificación: ¿existe un plan actualizado?, ¿hay responsables?, ¿se identificaron riesgos?, ¿se aprobaron los cambios? Para requisitos: ¿existe ERS?, ¿cada requisito está identificado?, ¿se revisaron y aprobaron?, ¿se controlan modificaciones?

El **informe** documenta auditoría, participantes, alcance, resultados y solicitudes de corrección. La cátedra clasifica los hallazgos como:

- **Buena práctica:** desempeño o solución superior a lo esperado; puede utilizarse como referencia.
- **Desviación:** incumplimiento de un criterio que requiere plan de acción.
- **Observación:** aspecto que convendría mejorar, sin requerir necesariamente un plan formal.

Las métricas pueden incluir esfuerzo y duración por auditoría, cantidad de desviaciones y distribución por áreas de proceso de CMMI. El número de hallazgos aislado no demuestra calidad ni efectividad del auditor.

7.6. Auditorías al Grupo de Calidad

La presentación distingue las **auditorías realizadas por el Grupo de Calidad** de proyecto, configuración funcional y física. No define explícitamente una categoría independiente denominada «auditoría al propio Grupo de Calidad». Si se aplica una auditoría al GAC, el objeto es el proceso de aseguramiento: se comprobaría independencia, planificación, competencias, evidencias, reportes, seguimiento de no conformidades y mejora. Esta explicación es una aplicación razonada de los principios generales, no una clasificación textual de la cátedra.

7.7. Ejemplo

Una auditoría del proceso de requisitos descubre cambios implementados sin aprobación. El auditor obtiene registros, clasifica el incumplimiento como desviación y documenta el hallazgo. El responsable propone registrar y aprobar los cambios pendientes y ajustar el procedimiento. El auditor comprueba nuevas evidencias antes de cerrar la desviación. La función principal fue evaluar el **cumplimiento del proceso**, no ejecutar las funciones del software.

8. Calidad de producto: planificación de pruebas

8.1. Qué es testing

El **testing** es un proceso que incluye actividades estáticas y dinámicas de planificación, preparación, ejecución y evaluación para proporcionar información sobre la calidad, detectar defectos y examinar la adecuación del software a las necesidades. La cátedra utiliza definiciones de IEEE e ISTQB. Myers, en *The Art of Software Testing*, enfatiza que una prueba exitosa puede ser aquella que descubre un defecto todavía desconocido.

El testing no «inyecta» calidad en un producto terminado ni garantiza que esté libre de defectos. Una prueba aprobada proporciona evidencia para las condiciones evaluadas. También es necesario reconocer que la ausencia de fallas observadas no demuestra ausencia total de fallas posibles.

8.2. Plan de pruebas (Test Plan)

La cátedra define la **planificación del testing** como la actividad de comprender objetivos del cliente y partes interesadas, metas del proyecto y riesgos de las pruebas. El Test Plan contiene cuatro componentes centrales:

Riesgos y objetivos. Definir qué problemas importan y qué se espera comprobar. Ejemplo: en un banco, la corrección de saldos y la seguridad tienen prioridad elevada.

Estrategia. Seleccionar niveles, tipos y técnicas; decidir frecuencia, automatización, ambientes y cobertura.

Recursos. Establecer personas, tiempos, herramientas, datos, infraestructura y ambientes.

Criterios de aceptación o salida. Definir condiciones para concluir una etapa: porcentaje de pruebas ejecutadas, resultados de pruebas críticas, severidad de defectos pendientes, costos y nivel de riesgo aceptado.

Los criterios de aceptación de una historia describen el comportamiento exigido por esa funcionalidad; los criterios de salida de testing determinan si existe evidencia suficiente para finalizar una etapa de pruebas. No deben confundirse.

8.3. Proceso completo de testing

La presentación muestra un ciclo de **planificación, identificación, especificación, ejecución y análisis de fallas**, con control durante todo el recorrido y decisiones sobre si se debe continuar probando.

Planificación y control: objetivos, riesgos, estrategia, recursos, criterios, seguimiento de resultados, cobertura y decisiones.

Identificación y especificación: examinar la base de pruebas, revisar requisitos, evaluar testabilidad, determinar datos, diseñar y priorizar casos, diseñar el ambiente.

Ejecución: preparar casos, datos, suites y entornos; automatizar lo necesario; identificar la versión; ejecutar, registrar resultados, comparar con los esperados y reportar problemas.

Evaluación y reporte: comparar resultados con criterios de aceptación, evaluar defectos pendientes, comunicar el estado, cerrar pruebas y extraer lecciones aprendidas.

Los **entregables** típicos son plan de testing, documento o repositorio de casos, registros de ejecución, reportes de incidentes e informe final. No todos requieren el mismo formato en todo proyecto.

8.4. Caso de prueba

Según las definiciones citadas por Toledo, un caso de prueba establece entradas, condiciones de ejecución y resultados esperados con un propósito específico. Puede incluir identificador, requisito, objetivo, precondiciones, datos, pasos, resultado esperado y resultado real.

Ejemplo — CP-01, autenticación correcta: precondición: usuario activo registrado. Datos: usuario y contraseña válidos. Pasos: abrir la pantalla, completar credenciales y confirmar. Resultado esperado: se inicia la sesión y se presenta la pantalla principal. Si el resultado real difiere, se analiza si existe defecto.

Una **condición de prueba** es un comportamiento o aspecto que debe verificarse. A partir de una condición se diseñan uno o varios casos. Un **ciclo de pruebas** es la ejecución del conjunto previsto sobre una versión identificada del software.

8.5. ¿Cuánto testing es suficiente?

El testing exhaustivo es imposible para sistemas reales con múltiples estados, entradas y configuraciones. La selección de pruebas requiere analizar los riesgos y costos. Los criterios de salida pueden considerar defectos por severidad, pruebas críticas ejecutadas, cobertura y restricciones del negocio. Debe quedar claro qué aspectos no fueron evaluados y cuál es el riesgo residual.

9. Niveles de prueba

Un **nivel de prueba** agrupa actividades según el alcance del software evaluado. La cátedra identifica cuatro niveles.

9.1. Unitarias

Comprueban componentes individuales después de su construcción: funciones, métodos, clases o unidades equivalentes. Normalmente utilizan acceso al código, entornos de desarrollo, frameworks de unit testing y depuradores. Por ejemplo, comprobar que una función que aplica 20 % de descuento a $100 produce $80. Se detectan y corrigen defectos cerca del código; no comprueban necesariamente la interacción con otros componentes.

9.2. Integración

Verifican la colaboración entre componentes que pueden funcionar bien aisladamente. Ejemplo: después de registrar una venta, comprobar que se actualiza correctamente el inventario. Se buscan errores de interfaces, contratos, intercambio de información y dependencias.

Top-Down: se integran primero componentes superiores y después inferiores; pueden emplearse *stubs* para simular componentes ausentes. **Bottom-Up:** se integran desde los componentes inferiores; pueden utilizarse *drivers* para ejecutar unidades sin sus niveles superiores. La cátedra recomienda combinar estrategias cuando resulta conveniente.

9.3. Sistema

Evalúan la aplicación integrada como un todo en un ambiente lo más parecido posible a producción. Comprenden pruebas de requisitos funcionales y características no funcionales: recuperación, seguridad, estrés, rendimiento y otras. Ejemplo: ejecutar una compra completa y comprobar autenticación, carrito, pago, inventario y comprobante.

9.4. Aceptación

Buscan determinar si el producto resulta adecuado para clientes y usuarios. No tienen como único objetivo encontrar defectos: aportan evidencia para aceptar el producto. **Alfa:** en condiciones controladas con usuarios; **Beta:** uso por usuarios en un entorno real o próximo al real, según las políticas del producto.

Nivel	Pregunta	Ejemplo
Unitario	¿Funciona este componente?	Cálculo de una comisión.
Integración	¿Funcionan juntos?	Venta e inventario.
Sistema	¿Funciona el conjunto?	Proceso de compra completo.
Aceptación	¿Satisface el negocio?	Usuario realiza una tarea real.

10. Tipos de pruebas

Pruebas funcionales: verifican funciones, reglas y comportamientos especificados. Pueden basarse en requisitos o procesos de negocio. Ejemplo: comprobar que no se autorice una operación cuando faltan fondos.

Pruebas no funcionales: evalúan características como rendimiento, fiabilidad, seguridad, usabilidad, portabilidad, instalación, configuración y mantenibilidad.

Rendimiento: tiempos de respuesta y consumo de recursos. **Carga:** comportamiento bajo trabajo o concurrencia previstos. **Estrés:** comportamiento ante condiciones más exigentes que las normales. **Volumen:** procesamiento con grandes cantidades de datos.

Regresión: comprueba que las modificaciones no afecten funciones que anteriormente estaban correctas. No equivale a **reprueba**, que confirma específicamente la corrección de un defecto.

Smoke Test: verificación breve de estabilidad básica de una versión antes de emprender pruebas más detalladas. Si el sistema no inicia, no conviene comenzar pruebas extensas de sus funciones.

Pruebas de interfaz: verifican acciones e interacciones desde pantallas, navegación y formularios. **Pruebas de configuración:** comprueban diferentes configuraciones de software, dispositivos, redes u otros componentes.

Estáticas: examinan productos sin ejecutar software, como revisión de requisitos o inspección de código. **Dinámicas:** ejecutan el programa y observan su comportamiento.

Distinción clave: un **nivel** indica el alcance del producto probado; un **tipo** indica el aspecto que se evalúa. Una prueba funcional puede ser unitaria, de integración, de sistema o de aceptación.

11. Técnicas de prueba

11.1. Caja negra

Se diseñan a partir de requisitos, especificaciones, reglas y comportamientos esperados, sin necesitar conocer el código interno.

Partición de equivalencias. Agrupa datos en clases con comportamiento esperado equivalente. Si se aceptan edades de 18 a 65 inclusive: menores de 18 (inválida), 18 a 65 (válida) y mayores de 65 (inválida). Pueden seleccionarse representantes 15, 30 y 70.

Análisis de valores límite. Se concentra en las fronteras de las clases, donde suelen aparecer defectos de comparación. Para edades 18-65 probar 17, 18, 19, 64, 65 y 66. Detectaría, por ejemplo, una condición que excluye incorrectamente 18.

Tablas de decisión. Representan combinaciones de condiciones y acciones. Si un cupón debe ser válido **y** la compra superar un mínimo, pueden probarse las cuatro combinaciones de ambas condiciones para verificar el descuento.

Pruebas basadas en casos de uso. Derivan casos a partir del flujo principal y alternativos de escenarios de negocio. En «Iniciar sesión», evaluar credenciales correctas, incorrectas y cuenta bloqueada según requisitos.

Máquinas de estado. Verifican estados y transiciones válidas o inválidas. En un pedido: Pendiente -> Pagado -> Enviado; comprobar que Cancelado no vuelva indebidamente a Enviado.

Grafos causa-efecto. Modelan relaciones lógicas entre entradas y resultados y sirven como apoyo para derivar combinaciones.

Pairwise y combinatorias. Seleccionan casos que cubran interacciones entre pares de valores, reduciendo el conjunto respecto de probar todas las combinaciones. No garantizan descubrir fallas que dependan de tres o más parámetros.

Matriz CRUD. Relaciona entidades con operaciones Crear, Leer, Actualizar y Eliminar, para identificar operaciones a probar.

11.2. Caja blanca

Utilizan conocimiento de la estructura interna del código. Se relacionan con criterios de cobertura:

Cobertura de sentencias: ejecutar cada instrucción relevante al menos una vez.

Cobertura de decisiones/ramas: ejecutar los resultados verdadero y falso de las decisiones aplicables.

Cobertura de condiciones: comprobar resultados verdadero/falso de condiciones individuales dentro de decisiones compuestas.

Cobertura de decisión-condición: combinar los criterios anteriores.

Cobertura de caminos: considerar recorridos posibles del flujo de control. Puede ser inviable cubrir todos los caminos de programas con ciclos.

Cobertura múltiple de condiciones: evaluar combinaciones de condiciones individuales, con mayor costo combinatorio.

Ejemplo: en `if usuario_activo and saldo_suficiente:` no alcanza con una prueba en la que ambas condiciones son verdaderas. Para distintos objetivos de cobertura deben diseñarse casos que produzcan resultados alternativos y ejerciten cada condición.

Advertencia: 100 % de cobertura de sentencias no significa ausencia de defectos; un caso puede ejecutar código sin comprobar adecuadamente el resultado.

11.3. Basadas en experiencia

Adivinanza de defectos (Error Guessing). Seleccionar entradas y situaciones propensas a falla a partir del conocimiento del producto y de problemas recurrentes. Por ejemplo, archivo vacío, datos duplicados o conexión interrumpida.

Testing exploratorio. Diseñar, ejecutar y aprender simultáneamente, con objetivos y límites definidos. No equivale a probar al azar. Una sesión puede investigar qué ocurre si se interrumpe el envío de un formulario y registrar hallazgos, hipótesis y nuevos casos.

Familia	Base	Ejemplos
Caja negra	Requisitos y comportamiento externo	Equivalencias, límites, tablas, estados, pairwise
Caja blanca	Estructura interna	Sentencias, decisiones, condiciones y caminos
Experiencia	Conocimiento y exploración	Error guessing, sesiones exploratorias

Las técnicas se complementan: ninguna es suficiente por sí sola.

12. Herramientas para probar software

Frameworks unitarios: JUnit es el ejemplo de la cátedra para Java; facilita aserciones y ejecución repetible.

Automatización funcional: Selenium aparece en la bibliografía de Toledo como herramienta de pruebas web. Registrar clics con *record and playback* no reemplaza la necesidad de verificar resultados y mantener scripts.

Diseño combinatorio y basado en modelos: CTWeb aparece en Toledo como herramienta para seleccionar combinaciones y trabajar con modelos.

Medición de cobertura: herramientas como Cobertura, citada por Kniberg, informan sobre código ejecutado por las pruebas. Cobertura no equivale a calidad asegurada.

Rendimiento y carga: herramientas para generar concurrencia, medir tiempos y analizar consumo de recursos.

Gestión de pruebas y defectos: registros de casos, ejecuciones, versiones, incidencias y su seguimiento.

Integración continua: herramientas de construcción y automatización de pipelines, como los ejemplos Maven y QuickBuild de Kniberg, permiten ejecutar verificaciones después de cambios.

Qué automatizar: pruebas repetitivas, riesgos importantes y regresiones frecuentes, considerando costo de creación, mantenimiento, fiabilidad y frecuencia de ejecución. La automatización no elimina exploración ni usabilidad.

13. Los siete principios generales de testing

La presentación de la cátedra reúne siete principios fundamentales:

1. **El testing muestra la presencia de defectos, no su ausencia absoluta.**
2. **El testing exhaustivo es imposible.** Deben seleccionarse escenarios relevantes.
3. **Testing temprano.** Detectar problemas cerca de su origen reduce retrabajo.
4. **Agrupamiento de defectos.** Determinados módulos suelen concentrar problemas.
5. **Paradoja del pesticida.** Repetir siempre los mismos tests sin cambiarlos puede dejar de descubrir defectos nuevos.
6. **El testing depende del contexto.** No sirve la misma estrategia para todos los productos.
7. **Falacia de ausencia de errores.** Un software puede tener pocos defectos técnicos y ser inútil para los usuarios.

---

14. Revisiones técnicas de software

14.1. Definición, propósito y diferencia con pruebas

Las **revisiones técnicas** son actividades de verificación estática que examinan productos de trabajo para identificar defectos, omisiones, inconsistencias, incumplimiento de estándares y otros problemas de calidad **sin ejecutar el programa**. Se aplican a requisitos, diseños, código, planes de proyecto, planes de pruebas y casos de prueba, incluso cuando el sistema todavía está incompleto.

La cátedra ubica su origen formal en las inspecciones introducidas por **Michael Fagan en 1976** para detectar defectos tan cerca como sea posible del momento en que se generan. La motivación es reducir propagación y retrabajo.

Ventajas: detectar errores temprano, revisar versiones incompletas, evaluar atributos distintos de la funcionalidad, favorecer intercambio de conocimiento y uniformidad.

Dificultades: capacitación, preparación, coordinación de participantes, esfuerzo de registro y costos iniciales. No deben convertirse en instrumentos para evaluar el rendimiento individual de los desarrolladores.

14.2. Walkthroughs o recorridas

Un **Walkthrough** es un análisis estático relativamente informal en el que un diseñador o programador guía a participantes a través de un producto de trabajo. Se realizan preguntas y comentarios acerca de defectos, estándares o problemas de diseño. Puede exigir poca preparación y proporcionar feedback rápido; normalmente cuenta con pocas métricas y menor control formal del proceso.

Ejemplo: un desarrollador explica una sección de código a sus compañeros, que detectan que el proceso no contempla una cancelación. Se registra la observación y se acuerda corregirla.

14.3. Inspecciones formales

Una **inspección** es una revisión estructurada con participantes definidos, preparación individual, checklist, reunión, registro, corrección, seguimiento y mediciones. Busca detectar y remover defectos de manera sistemática. No tiene como propósito resolver todos los problemas durante la reunión, aprobar automáticamente el producto ni juzgar a las personas.

Aspecto	Walkthrough	Inspección
Formalidad	Baja	Alta
Participantes	Flexibles	Roles específicos
Preparación	Poca o variable	Individual y planificada
Herramientas	Comentarios y notas	Checklists, registros, métricas
Objetivo destacado	Feedback y comprensión	Detección sistemática de defectos
Seguimiento	Menos estructurado	Formal y verificable

14.4. Roles de la inspección

Autor: crea o mantiene el producto revisado, lo entrega, realiza retrabajo e informa defectos y esfuerzo. **Moderador:** organiza, dirige, distribuye el material, controla la dinámica y verifica correcciones. **Lector:** recorre el producto en la reunión. **Anotador:** documenta los hallazgos. **Inspector:** prepara individualmente el material, busca defectos y registra su tiempo.

La cátedra recomienda que el moderador entregue material a los inspectores con **48 horas de anticipación** para posibilitar una preparación real.

14.5. Seis etapas del proceso de inspección de Fagan

1. **Planificación:** seleccionar equipo, material, alcance y calendario.
2. **Visión general:** explicar el contexto y características del producto.
3. **Preparación:** cada inspector analiza individualmente y reúne posibles defectos.

4. **Reunión:** se presentan hallazgos, se contrastan con el material, se descartan falsos positivos y se registran defectos. No se utiliza para resolverlos todos.
5. **Corrección:** el autor modifica el producto de trabajo.
6. **Seguimiento:** el moderador verifica correcciones, recopila datos y decide si se requiere reinspección.

Las decisiones de salida pueden ser **aceptar sin modificaciones, rechazar o aceptar provisionalmente**. El informe debe establecer qué se revisó, quiénes participaron, qué defectos se encontraron y cuáles fueron las conclusiones.

La cátedra presenta recomendaciones operativas para el modelo convencional: duración máxima de reunión de **dos horas**; tamaños de aproximadamente **250 líneas de código** o **280 líneas de texto** por inspección. Son valores orientativos del procedimiento estudiado, no reglas universales.

14.6. Técnicas y herramientas de revisión

Checklists: preguntas organizadas que guían la detección. Para requisitos: ¿son claros, consistentes, completos y verificables? Para código: ¿se contemplan errores y límites?, ¿se respetan estándares?, ¿las interfaces son coherentes?

Guías de lectura: indican cómo recorrer y evaluar un producto; pueden asignar diferentes perspectivas según el conocimiento de los revisores.

Plantillas y registros: documentan participantes, material, resultados, defectos, decisiones y tiempos.

Selección de revisores: según la cátedra, el documento de arquitectura puede revisarse con arquitectos, analistas, líderes y testers; una ERS, con analistas, diseñadores, testers y representantes del negocio; el código, con programadores y perfiles complementarios.

Herramientas de soporte: sistemas de comparación de versiones, revisión de cambios, registro de problemas, control documental y análisis estático automatizado. La cátedra no prescribe una plataforma específica.

14.7. Métricas de inspección

Densidad de defectos: defectos encontrados / tamaño revisado. Ejemplo: 10 defectos en 200 líneas = 0,05 defectos/línea.

Defectos totales: defectos mayores + defectos menores, según la clasificación de la cátedra.

Esfuerzo de inspección: planificación + preparación + reunión + retrabajo.

Esfuerzo por defecto: esfuerzo total de inspección / defectos encontrados. Ejemplo: 5 horas-persona / 10 defectos = 0,5 horas-persona por defecto.

Porcentaje de reinspecciones: cantidad de reinspecciones / cantidad de inspecciones × 100.

El resultado depende del contexto y tipo de material. Encontrar muchos defectos puede reflejar un material problemático o una revisión muy efectiva; por sí solo no permite concluir la calidad de los participantes.

14.8. Clasificación de defectos y recomendaciones

Mayores: pueden provocar una falla operacional o resultado inesperado; por ejemplo, un error lógico que permite pagos incorrectos.

Menores: no necesariamente provocan fallas, pero afectan claridad, estándares o mantenimiento.

Cosméticos: ortografía, tipeo, formato o problemas documentales secundarios.

Reglas esenciales: **revisar el producto, no al productor; respetar agenda; evitar debates prolongados y resolución durante la reunión; registrar hallazgos; limitar participantes; preparar material; utilizar listas; entrenar a revisores y aprender de revisiones anteriores**.

Diferencia con auditoría: la revisión busca defectos del contenido; la auditoría evalúa el cumplimiento de criterios y procesos con evidencias objetivas. **Diferencia con pruebas dinámicas:** la revisión no necesita ejecutar el programa.

15. Testing en ambientes ágiles

15.1. Concepto

El **testing ágil** integra el testing a todo el proceso de desarrollo y lo convierte en responsabilidad del equipo completo. Las pruebas no son exclusivamente una fase final ni actividad de una persona separada que recibe el código después de programado. Además de descubrir defectos, permiten aclarar requisitos, orientar la implementación y proporcionar feedback temprano.

En el enfoque tradicional organizado por fases, el testing puede concentrarse después de la programación. En Agile, desarrollo y pruebas se realizan durante iteraciones, comparten información y verifican cada incremento. Esto no significa ausencia de planificación: se necesitan criterios, ambientes, automatización, estrategia de riesgos y disciplina.

15.2. Los cinco valores del Manifiesto de Testing Ágil de la cátedra

1. **Pruebas durante el desarrollo** sobre pruebas exclusivamente al final.
2. **Prevenir defectos** sobre limitarse a encontrar defectos.
3. **Entender lo que se está probando** sobre comprobar mecánicamente funciones.
4. **Construir un mejor sistema** sobre «romper» el sistema como fin en sí mismo.
5. **Responsabilidad de todo el equipo por la calidad** sobre responsabilidad exclusiva del tester.

No significa abandonar la búsqueda de fallas: significa utilizarla para mejorar el producto y prevenir nuevos defectos.

15.3. Nueve principios de Testing Ágil de la cátedra

1. El testing se mueve hacia adelante en el proyecto. Revisar historias y criterios antes de codificar.

2. Testing no es una fase. Acompaña diseño, programación, integración y entrega.

3. Todos hacen testing. La responsabilidad es compartida y los especialistas aportan habilidades complementarias.

4. Reducir la latencia del feedback. Obtener resultados pronto para corregir con el contexto vigente.

5. Las pruebas representan expectativas. Expresan resultados que el cliente necesita.

6. Mantener el código limpio y corregir defectos rápidamente. Prevenir acumulación de deuda técnica.

7. Reducir la sobrecarga documental de las pruebas. Conservar documentación útil sin burocracia innecesaria.

8. Las pruebas son parte del Done. Una historia no está completa solo porque está programada.

9. Pasar de probar al final a desarrollo conducido por pruebas. Utilizar TDD y ATDD, según corresponda, para guiar el trabajo.

15.4. Diez principios para testers ágiles de Crispin y Gregory

El capítulo 2 de *Agile Testing* presenta una lista diferente, orientada a la actitud y la colaboración: proporcionar feedback continuo; entregar valor al cliente; facilitar comunicación cara a cara; tener coraje; mantener simplicidad; mejorar continuamente; responder al cambio; autoorganizarse; centrarse en las personas; y disfrutar del trabajo. No debe confundirse con los nueve principios de las diapositivas: son clasificaciones complementarias.

15.5. Seis prácticas concretas de la cátedra

1. **Pruebas unitarias y de integración automatizadas**, repetidas con frecuencia.
2. **Regresión automatizada de sistema**, para detectar efectos de cambios recientes.
3. **Pruebas exploratorias**, para investigar problemas no previstos.
4. **TDD**, que guía la implementación de unidades mediante pruebas.
5. **ATDD**, que guía el desarrollo con ejemplos y pruebas de aceptación del negocio.
6. **Control de versiones de pruebas junto con código**, para mantener consistencia y reproducibilidad.

15.6. TDD y ATDD: diferencias

TDD: *Test-Driven Development*. **Red - Green - Refactor**: prueba que falla, código mínimo que pasa, mejora del diseño preservando el comportamiento. Ayuda especialmente al diseño y calidad interna.

ATDD: *Acceptance Test-Driven Development*. El Product Owner, testers y programadores acuerdan ejemplos y criterios de aceptación antes de implementar una historia. Las pruebas expresan comportamientos observables del negocio y pueden automatizarse.

Ejemplo: «Los clientes Premium reciben un descuento del 10 % en compras elegibles». Con ATDD se definen situaciones de aceptación: cliente Premium elegible, cliente estándar y compra no elegible. Con TDD el desarrollador implementa y verifica pequeñas funciones del cálculo. Las prácticas operan en niveles complementarios.

15.7. Rol del tester en equipos ágiles

El tester participa en el análisis de historias, revisión de criterios de aceptación, identificación de riesgos, diseño y automatización de pruebas, exploración, gestión de defectos y evaluación de Done. Colabora con Product Owner, desarrolladores y equipos que administran ambientes o infraestructura.

Whole-Team Approach: la responsabilidad de calidad es de todas las personas. Crispin y Gregory proponen la colaboración estrecha y destacan el *Power of Three*: negocio, programación y testing conversan sobre una historia para compartir reglas, ejemplos y riesgos. El tester deja de actuar como policía de calidad externo al desarrollo y contribuye directamente a construir un producto de valor.

15.8. Los cuatro cuadrantes de Testing Ágil

Brian Marick propuso una clasificación retomada por Crispin y Gregory. Dos ejes: **orientación al negocio vs. a la tecnología**, y **apoyo al desarrollo vs. crítica/evaluación del producto**.

Cuadrante	Orientación	Finalidad	Ejemplos
Q1	Tecnología	Apoyar y guiar	Unitarias, componentes, TDD
Q2	Negocio	Apoyar y guiar	Ejemplos, pruebas funcionales, historias, ATDD, prototipos
Q3	Negocio	Evaluar críticamente	Exploratorias, usabilidad, UAT, Alfa/Beta
Q4	Tecnología	Evaluar críticamente	Rendimiento, carga, seguridad, recuperación, otros atributos

Q1: pruebas automatizadas, rápidas, orientadas al código y calidad interna. **Q2:** comportamientos comprensibles para negocio que guían desarrollo y pueden automatizarse. **Q3:** investigación del producto desde la perspectiva del usuario; predomina la evaluación humana. **Q4:** características técnicas del producto y riesgos no funcionales, a menudo con herramientas especializadas.

Los cuadrantes no son etapas ni indican orden cronológico. Por ejemplo, un sistema crítico de gran carga puede necesitar Q4 desde la primera iteración. También pueden existir pruebas de aceptación en distintos cuadrantes según su finalidad.

15.9. Pirámide de Testing Ágil

La pirámide ayuda a planificar automatización: una **base amplia** de pruebas unitarias y componentes; un **nivel medio** de servicios, API y aceptación; una **cima reducida** de pruebas GUI. En general, las verificaciones de bajo nivel son rápidas y menos costosas de mantener, mientras que las GUI pueden ser lentas y frágiles frente a cambios de presentación.

La cátedra compara una pirámide ágil con la pirámide tradicional invertida, donde predominan pruebas GUI. Las proporciones 80-90 % que aparecen visualmente son ilustrativas, no objetivos numéricos obligatorios.

Automatización y exploración: no son excluyentes. Las pruebas automatizadas de regresión proporcionan seguridad para modificar y refactorizar; las exploratorias permiten descubrir problemas y necesidades no previstos.

15.10. Testing dentro de una iteración

Antes de desarrollar: Product Owner y equipo acuerdan historia, ejemplos, riesgos y criterios de aceptación.

Durante el desarrollo: se realizan pruebas unitarias, integración y otras verificaciones; tester y desarrolladores comparten información y corrigen defectos.

Al completar la historia: se verifican criterios de aceptación y condiciones de Done, se realizan pruebas relevantes y se considera el incremento listo.

En la revisión: los interesados proporcionan feedback sobre el resultado real.

En la retrospectiva: el equipo analiza problemas de testing, como pruebas lentas, ambientes inestables o criterios ambiguos, y acuerda mejoras.

15.11. Automatización y cambios culturales

La presentación identifica beneficios como ejecución rápida, reuso, mayor cobertura, precisión, alcance y posible reducción de costos. Pero automatizar exige desarrollar, mantener y revisar scripts, datos y ambientes.

La cultura debe pasar de áreas aisladas a colaboración: programadores también prueban; testers participan antes y durante desarrollo; negocio ayuda a expresar criterios verificables; equipos comparten información y responsabilidad.

Un cambio de denominación o la incorporación de herramientas no garantiza agilidad. Sin automatización confiable, integración frecuente y comunicación, las pruebas pueden acumularse al final de cada Sprint.

15.12. Siete factores de éxito según Crispin y Gregory

1. Enfoque de equipo completo.
2. Mentalidad de testing ágil.
3. Automatización de pruebas de regresión.
4. Proporcionar y obtener feedback constante.
5. Base de prácticas técnicas: CI, ambientes, trabajo incremental y deuda técnica controlada.
6. Colaboración estrecha con clientes y negocio.
7. Mantener visión global del producto, no solo de historias aisladas.

Idea clave: el objetivo no es «probar más» ni «entregar más rápido a cualquier costo», sino entregar software valioso y de calidad con ciclos de aprendizaje cortos.

16. Desarrollo ampliado: razonamiento de calidad de proceso y de producto

16.1. Calidad no equivale a «ausencia de errores»

La presentación de PPQA define calidad como el conjunto de aspectos y características de un producto o servicio relacionados con su capacidad para satisfacer necesidades **manifiestas e implícitas**. La noción es **relativa** al usuario, al momento y al contexto. Un sistema sin defectos funcionales conocidos puede ser de baja calidad si no sirve para realizar la tarea real o si no satisface condiciones importantes de rendimiento, seguridad o accesibilidad.

La **visión trascendental** asocia calidad con excelencia difícil de formalizar; la **visión del producto** con atributos observables; la **visión del usuario** con satisfacción de necesidades; la **visión de manufactura** con conformidad con especificaciones; la **visión del valor** con beneficios en relación con el costo. Estas visiones no son sinónimos: si alguien dice «cumplimos todos los requisitos», está enfatizando conformidad; si otra persona dice «el usuario abandona la aplicación porque es difícil de utilizar», enfatiza adecuación al uso.

Calidad programada es la prevista; **realizada**, la obtenida; **necesaria**, la que efectivamente requiere el usuario. La superposición de las tres es la situación buscada. Cuando se programa una funcionalidad innecesaria puede existir desperdicio; cuando una necesidad no fue programada puede existir insatisfacción; cuando lo programado no se realiza correctamente puede existir incumplimiento. El esquema de la cátedra está en las páginas 195 y 196 del PDF compilado.

Ejemplo triple: la universidad necesita inscripciones sin duplicaciones (necesaria). El equipo planifica un control de unicidad (programada). Si el producto efectivamente impide duplicados bajo las condiciones

relevantes (realizada), existe coincidencia. Si el equipo planifica un campo adicional que nadie utiliza, agrega trabajo que no representa necesariamente valor; si olvida un caso crítico, la planificación es incompleta.

16.2. Calidad interna, externa y en uso

La **calidad interna** considera propiedades de diseño y estructura del software, como modularidad, comprensibilidad y facilidad de modificación. La **calidad externa** considera comportamientos visibles al ejecutar el sistema. La **calidad en uso** observa lo que personas concretas consiguen hacer con el producto en situaciones reales. Las tres perspectivas se conectan: un diseño interno deficiente puede dificultar el mantenimiento y terminar afectando la calidad externa. Un producto técnicamente correcto no necesariamente consigue que el usuario complete su tarea.

La presentación utiliza el **modelo ISO/IEC 25010:2011**, con ocho características de calidad de producto: adecuación funcional, eficiencia de desempeño, compatibilidad, usabilidad, fiabilidad, seguridad, mantenibilidad y portabilidad. Conviene conocer subcaracterísticas que aparecen en el cuadro: completitud, corrección y pertinencia funcional; comportamiento temporal, utilización de recursos y capacidad; coexistencia e interoperabilidad; aprendizaje, operabilidad, protección ante errores de usuario y accesibilidad; disponibilidad, tolerancia a fallos y recuperabilidad; confidencialidad, integridad, no repudio, autenticidad y responsabilidad; modularidad, reusabilidad, analizabilidad, modificabilidad y capacidad de prueba; adaptabilidad, instalabilidad y reemplazabilidad.

El modelo de **calidad en uso** no es una lista intercambiable con las ocho características del producto. También aparece la **calidad de datos** como objeto diferenciado de la familia ISO/IEC 25000. Las otras clasificaciones presentadas por la cátedra, como McCall y Barbacci/SEI, muestran que los modelos pueden agrupar atributos de manera distinta y que optimizar uno de ellos no garantiza un óptimo global.

16.3. Diez principios de calidad: cómo justificarlos

«La calidad debe estar embebida» significa que debe influir en requisitos, diseño, código y verificación. «Es un esfuerzo de todos» implica responsabilidad compartida, sin eliminar la especialización. «Las personas son la clave» exige formación y participación. «Se necesita patrocinio gerencial» reconoce que recursos y prioridades dependen de decisiones de dirección. «Liderar con el ejemplo» evita que las prácticas se conviertan en exigencias que las autoridades no cumplen.

«No se puede controlar lo que no se mide» exige datos pertinentes, aunque medir no garantiza una decisión acertada. «Empezar por lo básico» reduce el riesgo de implantar burocracia compleja. «Planificar SQA» significa definir responsabilidades y actividades antes de que aparezcan problemas. «Más pruebas no implica más calidad» recuerda la importancia de cobertura y relevancia. «Debe ser razonable para el negocio» exige proporcionalidad entre riesgo, beneficios y esfuerzo.

Ventajas de un enfoque apropiado: prevención de defectos, menor retrabajo, mejor satisfacción de usuarios, mayor previsibilidad, mantenimiento más sencillo, control de riesgos y aprendizaje organizacional. **Costos y dificultades**: capacitación, coordinación, infraestructura, tiempo de inspecciones, resistencia al cambio, medición imperfecta, posible burocracia y falsa seguridad. Una práctica de calidad solo agrega valor si ayuda a reducir riesgos o mejorar resultados de manera justificable.

16.4. QA, QC, Testing, V&V: un ejemplo que los distingue

Supongamos que una empresa debe desarrollar un sistema de transferencias. **Planificar la calidad** implica fijar atributos y criterios: seguridad, rendimiento, confiabilidad y mecanismos de evaluación. **QA** puede establecer y evaluar el procedimiento de revisión de requisitos, la gestión de cambios y las actividades que deben cumplirse. **QC** puede controlar que esas reglas y criterios efectivamente se satisfagan, mediante

revisiones, mediciones y evaluaciones. **Testing** puede ejecutar escenarios de transferencias válidas, inválidas o bajo carga para obtener información sobre el producto.

Verificación: se comprueba que el sistema cumple las especificaciones correspondientes. **Validación:** se comprueba que resuelve el problema real y resulta adecuado al uso. Un conjunto grande de pruebas automatizadas no reemplaza la validación con usuarios y partes interesadas.

17. Profundización de modelos: qué evalúan y qué no evalúan

17.1. CMMI-DEV v1.3: representaciones y organización

La bibliografía de PPQA desarrolla **CMMI-DEV versión 1.3**. La **representación por etapas** organiza la mejora en niveles de **madurez** de la organización: 1 Inicial, 2 Administrado, 3 Definido, 4 Administrado cuantitativamente y 5 Optimizado. La **representación continua** considera la **capacidad** de áreas individuales con niveles de 0 a 5, según la versión estudiada. No se deben sumar o equiparar mecánicamente los niveles de ambas representaciones.

En el **nivel 2**, las áreas de gestión del proyecto incluyen administración de requisitos, planificación, monitoreo y control, y gestión de proveedores. Las áreas de soporte incluyen **Medición y Análisis (MA), Aseguramiento de Calidad de Proceso y Producto (PPQA) y Administración de Configuración (CM).** El objetivo es que las actividades de gestión básica sean planificadas y controladas.

En el **nivel 3** se definen procesos organizacionales y aparecen áreas de ingeniería (desarrollo de requisitos, solución técnica, integración del producto, verificación y validación), procesos organizacionales (foco, definición y capacitación), gestión integrada, riesgos y análisis de decisiones. Los procesos se establecen de forma compartida y se adaptan al contexto del proyecto.

En el **nivel 4** se utilizan mediciones y métodos cuantitativos para comprender y gestionar desempeño organizacional y de proyectos. En el **nivel 5** se incorporan mejoras sistemáticas de desempeño y análisis causal de problemas. El progreso no se reduce a tener más documentos: se espera una forma de trabajo efectivamente aplicada y respaldada por evidencias.

Ejemplo de clasificación: un equipo posee un registro de cambios, pero lo actualiza solo cuando alguien se acuerda: puede estar ejecutando parte del proceso sin gestionarlo de forma consistente. Si las solicitudes, responsables, decisiones y versiones están controlados, existen características de gestión. Si toda la organización utiliza un proceso definido y adaptable, se avanza hacia una situación compatible con mayor institucionalización. No se puede asignar formalmente un nivel de CMMI con ese ejemplo aislado.

CMMI y Agile: las prácticas ágiles no son una excusa para omitir control de requisitos, configuración o evaluación; tampoco exige CMMI imponer una cantidad universal de documentos. Lo relevante es interpretar las prácticas y demostrar su cumplimiento con evidencias útiles. Los mecanismos concretos pueden diferir entre organizaciones.

17.2. SPICE y límites de la fuente

El material de la cátedra **menciona SPICE, pero no desarrolla sus niveles**. La presentación de los seis niveles clásicos asociados con ISO/IEC 15504 en esta guía es un **complemento conceptual**, no un contenido extensamente explicado en las diapositivas. El foco está en evaluar **procesos individuales** mediante atributos de capacidad.

En el modelo clásico: **0 Incompleto**, no alcanza su propósito; **1 Realizado**, alcanza su propósito; **2 Gestionado**, se planifica, supervisa y controlan productos de trabajo; **3 Establecido**, el proceso está definido y desplegado; **4 Predecible**, se gestiona cuantitativamente; **5 En optimización**, se mejora de forma

sistemática. Los atributos corresponden a desempeño (nivel 1); gestión del desempeño y de productos de trabajo (nivel 2); definición y despliegue (nivel 3); medición y control (nivel 4); innovación y optimización (nivel 5).

La capacidad de un proceso particular no equivale automáticamente a la madurez global de la organización. CMMI y SPICE pueden orientar mejoras, pero sus modelos de evaluación no deben confundirse.

17.3. ISO: evitar la confusión entre normas de gestión, proceso y producto

ISO 9001 establece requisitos de un sistema de gestión de calidad. **ISO/IEC 90003** orienta su aplicación al software. **ISO/IEC 12207** proporciona un marco de procesos del ciclo de vida del software. **ISO/IEC 25010:2011**, dentro de SQuaRE, proporciona modelos de calidad de producto y calidad en uso; **ISO/IEC 25012:2008** aborda calidad de datos. **ISO/IEC 15504** es la referencia histórica asociada con SPICE.

La diapositiva de la cátedra desarrolla directamente ISO/IEC 25000, ISO/IEC 25010:2011 e ISO/IEC 25012:2008; las explicaciones de ISO 9001, ISO/IEC 90003 e ISO/IEC 12207 son complementos. Una **certificación del sistema de gestión** no prueba que cada producto esté libre de defectos. Un **modelo de calidad de producto** tampoco establece por sí mismo un nivel de madurez organizacional.

17.4. IDEAL aplicado a una mejora concreta

El modelo **IDEAL** organiza una iniciativa de mejora en cinco fases: **Initiating**, establecer necesidad, apoyo e infraestructura; **Diagnosing**, evaluar el estado actual y descubrir problemas; **Establishing**, definir objetivos, prioridades y planes; **Acting**, implementar cambios y pilotos; **Learning**, analizar resultados y ajustar el enfoque.

Ejemplo: una organización observa demoras en pruebas. En *Initiating* acuerda actuar y obtiene apoyo; en *Diagnosing* descubre que los ambientes son inestables; en *Establishing* decide automatizar su configuración; en *Acting* ensaya el nuevo proceso con un equipo; en *Learning* compara tiempos y defectos y decide cómo mejorarlo o extenderlo. El modelo es una **secuencia para conducir la mejora**, no una escala de madurez.

18. Auditorías en profundidad: evidencia, roles y cierre de hallazgos

18.1. Auditoría no equivale a inspección técnica

La **auditoría** evalúa la conformidad de productos o procesos con requisitos, estándares y procedimientos mediante criterios y evidencia objetiva. La **inspección técnica** examina el contenido de un producto de trabajo y busca defectos. En una auditoría de proyecto se puede verificar que existan informes de inspección aprobados, sin repetir personalmente toda la inspección del diseño. En una inspección de código se pueden encontrar problemas técnicos aunque el procedimiento formal de auditoría nunca se haya ejecutado.

El material de la cátedra utiliza IEEE Std 1028-1988 como referencia para la definición de auditoría. El PACS (Plan de Aseguramiento de Calidad de Software) debe orientar qué auditorías se realizarán y quién será responsable.

18.2. Las tres auditorías descritas por la cátedra

Auditoría de proyecto: comprueba cumplimiento del proceso de desarrollo y coherencia entre productos. La presentación identifica relaciones entre interfaces y requisitos de la **ERS**, requisitos y **Plan de Verificación y Validación**, requisitos y diseño del **DDS**, y diseño y código. Un cambio de requisito debe propagarse de manera coherente a documentos, implementación y actividades de verificación.

Auditoría de configuración funcional: evalúa la conformidad del software construido con los requisitos funcionales. Puede incluir revisión de ejecutables, registros de pruebas, ERS y matriz de rastreabilidad. La pregunta principal es: «¿El producto implementa aquello que se especificó?».

Auditoría de configuración física: evalúa la correspondencia entre el software construido y la documentación técnica que lo describe. La pregunta principal es: «¿La versión que entregamos coincide con los artefactos y documentos identificados?».

Ejemplo: un producto tiene una nueva versión de la API, pero el manual de interfaces describe la anterior. La inconsistencia es especialmente relevante para la auditoría física. Si la API no implementa una operación exigida por la ERS, se trata de un problema funcional. Si los cambios fueron aprobados sin seguir el procedimiento obligatorio, existe un problema del proceso auditado.

18.3. Cuatro fases: objetivo, acciones y salida

Fase	Acciones relevantes	Salida
Preparación y planificación	Acordar fecha, objetivos, alcance, auditados; solicitar documentación	Plan y agenda de auditoría
Ejecución	Apertura, entrevistas, muestreo, revisión de registros y checklist	Evidencias y hallazgos preliminares
Análisis y reporte	Comparar con criterios, cierre, informe preliminar y final	Informe con resultados y desviaciones
Seguimiento	Planes de acción, responsables, fechas, comprobación	Desviaciones corregidas y cerradas

Fase de preparación: el auditor debe comprender qué se auditará, qué criterios se usarán y cuáles son las evidencias esperables. **Ejecución:** se solicitan ejemplos reales y se contrasta lo declarado con registros. **Análisis:** se distingue evidencia de interpretación, se clasifican los hallazgos y se explica su fundamento. **Seguimiento:** se verifica la eficacia de las acciones, no solo la declaración de que fueron realizadas.

El **Gerente de SQA** prepara el plan, asigna recursos y gestiona las no conformidades. El **auditor** evalúa objetivamente, documenta y hace seguimiento. El **auditado** proporciona evidencias, responde y propone/ejecuta acciones correctivas. El grupo de calidad debe tener suficiente independencia respecto de los proyectos evaluados; la cátedra recomienda evitar que dependa directamente del gerente del proyecto.

18.4. Checklist e informe: caso trabajado

Supongamos que se audita la administración de requisitos de un sistema académico. El checklist pregunta: «¿Existe una ERS?», «¿Los requisitos se identifican unívocamente?», «¿Se revisan y aprueban?», «¿Los cambios se controlan?», «¿La matriz de rastreabilidad está actualizada?».

Se selecciona una muestra de diez solicitudes de cambio y se revisan las evidencias. Ocho tienen aprobación documentada y dos no. Si el procedimiento exige aprobación previa, existe una **desviación** sustentada en dos registros concretos. El informe debería identificar criterio incumplido, evidencia, alcance y responsable del plan de acción. El auditado puede proponer investigar las dos solicitudes, corregir el estado documental y adaptar un control preventivo. En el seguimiento, el auditor revisa si el mecanismo realmente se utiliza.

La cátedra distingue **buena práctica** (procedimiento especialmente eficaz), **desviación** (incumplimiento que requiere plan de acción) y **observación** (situación mejorable sin obligación automática de plan formal). Una observación no es una desviación «pequeña» por definición: su tratamiento depende de los criterios de la auditoría.

18.5. ¿Qué significa «auditorías al grupo de calidad»?

Las diapositivas consultadas no desarrollan una categoría separada con ese título. Se deben distinguir dos interpretaciones: **auditorías realizadas por el GAC**, que evalúan procesos y productos de proyectos; y **auditorías del propio GAC**, en las que una evaluación suficientemente independiente examina si el grupo planifica, informa, actúa con objetividad y hace seguimiento de sus hallazgos. La segunda es una **aplicación general del concepto de auditoría**, no una clasificación independiente presentada por la cátedra.

19. Planificación de testing y diseño: casos completos

19.1. De la historia de usuario al plan y los casos de prueba

Historia: «Como estudiante quiero inscribirme a un examen para reservar mi lugar». El equipo identifica **riesgos**: inscripciones duplicadas, estudiantes sin condiciones académicas, cupos agotados, pérdida de datos ante fallas de red. Establece **objetivos**: comprobar reglas de inscripción, integridad de datos y comportamiento ante errores. Define **estrategia**: unitarias para reglas, integración con base de datos, funcionales de sistema, exploratorias y, si corresponde, carga. Determina **recursos**: ambiente, datos de estudiantes, roles y herramientas. Fija **criterios de aceptación y salida**: reglas críticas satisfechas y ausencia de defectos bloqueantes abiertos, según acuerdos del proyecto.

El proceso de testing de la cátedra contempla **planificación y control**, **identificación y especificación**, **ejecución**, y **evaluación de resultados y reporte**. El control no desaparece durante la ejecución: se revisan cobertura y riesgos para adaptar decisiones.

19.2. Caso de prueba formal

Campo	Valor ilustrativo
Identificador	CP-INS-03
Objetivo	Verificar que no se permita una inscripción duplicada
Requisito relacionado	RQ-INS-07
Precondición	Estudiante activo con una inscripción existente al mismo examen
Entrada	Nueva solicitud de inscripción para el mismo examen
Pasos	Ingresar como estudiante; buscar examen; intentar inscribirse nuevamente
Resultado esperado	La solicitud se rechaza o informa la condición sin crear un duplicado
Evidencia	Estado de la interfaz, registro de base de datos y log cuando corresponda

Un caso de prueba puede registrar también versión del software, ambiente, datos y resultados reales. La **base de pruebas** puede estar formada por ERS, reglas de negocio, diseños e historias. **Testabilidad** significa que el comportamiento está expresado de manera suficientemente precisa para ser verificado.

19.3. Niveles: cuatro preguntas diferentes

Unitario: «¿La regla individual funciona?». Por ejemplo, validar correlativas. **Integración:** «¿El módulo de inscripción se comunica correctamente con el módulo de alumnos y la base de datos?». **Sistema:** «¿El flujo completo de inscripción funciona en la aplicación?». **Aceptación:** «¿El personal y los estudiantes pueden realizar las tareas previstas y consideran aceptable la solución?».

En integración **Top-Down** se parte de componentes superiores y pueden utilizarse *stubs* para los inferiores todavía ausentes; en **Bottom-Up** se parte de componentes inferiores y pueden utilizarse *drivers* para invocarlos. La cátedra considera conveniente combinar ambos enfoques según contexto y componentes críticos.

Las pruebas **Alfa** se llevan a cabo en ambientes controlados con participación de usuarios; las **Beta** se realizan en condiciones reales de utilización por usuarios seleccionados. No deben confundirse los niveles con los tipos: seguridad, rendimiento y usabilidad pueden necesitarse en distintos niveles o instancias.

19.4. Funcionales, no funcionales, regresión, smoke y exploratorias

Las **funcionales** comprueban reglas de negocio y respuestas esperadas. Las **no funcionales** evalúan atributos como rendimiento, fiabilidad, seguridad, accesibilidad, instalación y portabilidad. **Carga** evalúa el sistema bajo demanda definida; **estrés** bajo condiciones exigentes que pueden sobrepasar los límites habituales; **volumen** frente a grandes cantidades de datos. **Regresión** comprueba que cambios recientes no afectaron comportamientos anteriormente correctos. **Reprueba** verifica específicamente la corrección de un defecto. **Smoke** realiza un control rápido de estabilidad básica antes de pruebas más detalladas.

Las **pruebas exploratorias** combinan diseño, ejecución y aprendizaje a medida que se investiga el producto. Pueden usar una misión y un tiempo de sesión. No son sinónimo de «hacer clic al azar». Complementan las pruebas guionadas y resultan especialmente valiosas cuando hay incertidumbre sobre comportamientos y uso real.

19.5. Partición de equivalencias y valores límite: caso resuelto

Regla: se admiten edades enteras **desde 18 hasta 65 inclusive**. Clases: menores a 18 (inválida), 18 a 65 (válida) y mayores a 65 (inválida). Representantes ilustrativos: 16, 30 y 70. Análisis de límites: **17, 18, 19, 64, 65 y 66**. Las pruebas de límite son importantes porque errores como usar `>` en vez de `>=` suelen manifestarse allí.

Una prueba adicional debe contemplar el tratamiento de entradas que no son edades enteras, datos vacíos y valores que no cumplen el tipo de dato requerido, cuando la interfaz o el servicio los admite. La partición necesita comprender tanto el dominio del requisito como las restricciones del sistema.

19.6. Tabla de decisión: combinaciones del negocio

Regla: un cupón se aplica cuando **está vigente**, **es válido para el producto** y la compra supera un mínimo. La tabla completa de las tres condiciones binarias contiene hasta ocho combinaciones. Una fila con las tres condiciones verdaderas debe aplicar descuento; cualquiera que no cumpla una condición obligatoria debe rechazarlo. Las tablas de decisión permiten identificar combinaciones que podrían olvidarse al redactar pruebas narrativas.

Los **casos de uso** permiten derivar flujos principales y alternativos. Las **máquinas de estados** permiten verificar transiciones, por ejemplo de Pedido a Pagado, Enviado o Cancelado y detectar transiciones inválidas. **Pairwise** busca cubrir pares de valores entre parámetros seleccionados, reduciendo la cantidad de combinaciones respecto de la enumeración completa, pero no garantiza detectar interacciones que

dependen de más de dos parámetros. La **matriz CRUD** permite comprobar operaciones crear, leer, modificar y eliminar sobre entidades y sus reglas.

19.7. Caja blanca: coberturas y limitaciones

La **cobertura de sentencias** busca ejecutar las instrucciones ejecutables; la **cobertura de decisiones** busca ejecutar los resultados posibles de cada decisión; la **cobertura de condiciones** examina resultados verdaderos y falsos de las condiciones elementales; la **cobertura de caminos** considera recorridos por el flujo de control. Son criterios diferentes y no garantizan por sí solos que las aserciones sean correctas ni que el producto satisfaga al usuario.

Ejemplo: `if (usuario_activo and saldo_suficiente): autorizar()`. Una prueba donde ambos son verdaderos y otra donde ambos son falsos pueden cubrir los dos resultados de la decisión, pero no necesariamente demuestran todos los efectos independientes de cada condición. Para analizar condiciones individuales se seleccionan combinaciones adicionales.

Las técnicas de caja negra se basan en especificaciones; las de caja blanca, en estructura interna; las basadas en experiencia, en conocimiento y aprendizaje del tester. La estrategia debe combinarlas: **ninguna técnica es completa**.

19.8. Herramientas: función y precaución

JUnit sirve como ejemplo de framework de pruebas unitarias; **Selenium**, como tecnología para automatizar interacción web; **herramientas de cobertura**, para analizar qué porciones del código se ejecutaron; **herramientas de gestión de defectos**, para registrar reproducibilidad, severidad y estado; **herramientas de rendimiento**, para simular carga y medir respuestas; **servidores de CI**, para ejecutar suites repetidas. En Toledo aparece **CTWeb** como apoyo a diseño combinatorio y modelos, mientras que Kniberg presenta Maven, QuickBuild y Cobertura en ejemplos de automatización.

Una herramienta no reemplaza el juicio del tester. El éxito no depende de contar miles de scripts sino de que proporcionen **información relevante y confiable**. Deben considerarse datos, mantenimiento de scripts, flakiness, costos de infraestructura y cobertura de los riesgos importantes.

20. Inspecciones y revisiones técnicas: desarrollo detallado

20.1. Revisiones estáticas y productos adecuados

Una **revisión técnica** examina documentos, diseños, requisitos, código o planes sin ejecutar el software. Su ventaja fundamental es la detección temprana: un requisito ambiguo puede corregirse antes de convertirse en código erróneo y múltiples casos de prueba defectuosos. Se puede revisar un producto incompleto y analizar atributos que una prueba dinámica no evidencia fácilmente, como claridad o mantenibilidad.

La cátedra presenta dos modalidades relevantes: **Walkthrough** o recorrida, generalmente informal y dirigida por el autor; e **inspección formal**, estructurada con roles, preparación, reunión y mediciones. La formalidad no hace superior toda revisión en cualquier contexto: debe elegirse el método considerando criticidad, costo y objetivo.

20.2. Roles y seis fases de inspección

Los cinco roles son **autor** (crea y corrige), **moderador** (planifica, dirige y sigue), **lector** (recorre el producto), **anotador** (registra hallazgos) e **inspector** (prepara y busca defectos). La cátedra recomienda distribuir el material con **48 horas de anticipación**.

Las seis etapas de inspección son **planificación**, **visión general**, **preparación individual**, **reunión de inspección**, **corrección** y **seguimiento**. En la reunión se consolidan defectos, se filtran falsos positivos y se registra una decisión. La cátedra menciona **aceptar sin modificaciones**, **rechazar** o **aceptar provisionalmente** el producto. Las soluciones se desarrollan en la fase de corrección; una reunión dominada por discusiones de implementación puede dejar sin inspeccionar el resto del material.

En el informe se documenta **qué se revisó, quién lo revisó, qué se descubrió y cuáles fueron las conclusiones**. La presentación utiliza como orientación reuniones de hasta dos horas, límites de tamaño del material y tasas de lectura o preparación. Son recomendaciones operativas del método descrito, no exigencias universales.

20.3. Checklists, guías y lectura por perspectivas

Una **lista de verificación** ayuda a revisar preguntas relevantes y defectos recurrentes: ambigüedad, completitud, consistencia, cumplimiento de estándares, testabilidad, interfaces, casos límite y trazabilidad. Una **guía de lectura** establece cómo recorrer el material. La cátedra sugiere participantes distintos según el documento: requisitos, arquitectura, código y plan de pruebas requieren combinaciones de analistas, diseñadores, testers, desarrolladores, líderes y representantes de negocio.

Una **revisión por perspectivas** es una explicación complementaria de cómo explotar conocimientos distintos: quien representa al usuario busca omisiones de negocio; el tester evalúa verificabilidad y casos límite; quien diseña examina coherencia técnica. No se debe presentar como una categoría formal independiente de la cátedra.

20.4. Costo y métricas de revisión

La cátedra identifica **costos de infraestructura** (capacitación, plantillas, guías, herramientas), **operativos** (tiempo individual y grupal, informes) y **adicionales** (preparación, coordinación, recolección de datos y mejora). Entre las métricas presenta **densidad de defectos**, **total de defectos mayores y menores**, **esfuerzo de inspección**, **esfuerzo por defecto** y **reinspecciones**.

Ejemplo: se revisan 250 líneas y se encuentran 15 defectos mayores y 5 menores. El total registrado es 20 y la densidad es **20/250 = 0,08 defectos por línea**, o **8 defectos cada 100 líneas**. Si el esfuerzo conjunto fue 10 horas-persona, el esfuerzo por defecto es **0,5 horas-persona**. Los resultados deben interpretarse en relación con el tamaño, complejidad y criterios de clasificación.

20.5. Reglas de conducta y propósito

La cátedra insiste en **revisar el producto, no al productor**. Una inspección no es un examen de desempeño individual ni una reunión para encontrar inmediatamente soluciones a todos los defectos. Conviene establecer agenda, limitar debates, registrar notas, disponer materiales con anticipación, utilizar listas y aprender de revisiones anteriores. El objetivo es encontrar problemas de manera organizada, reducir retrabajo y mejorar el proceso de trabajo.

21. Testing ágil: mapa completo del cambio de enfoque

21.1. Del control tardío a la prevención compartida

En testing ágil las pruebas dejan de ser una fase que comienza cuando termina programación. Se realizan desde el análisis de historias, mediante ejemplos y criterios de aceptación, y continúan con construcción, integración, exploración y aceptación. La calidad es responsabilidad del equipo completo. El tester aporta especialización y pensamiento crítico, pero no actúa como «policía de calidad» externo al proceso.

La presentación de la cátedra recoge cinco preferencias del **Manifiesto de Testing**: pruebas durante el desarrollo sobre pruebas al final; prevención de defectos sobre su descubrimiento tardío; comprensión del producto sobre verificación mecánica; construcción de un mejor sistema sobre «romperlo»; y responsabilidad del equipo completo sobre responsabilidad exclusiva del tester. Son preferencias de enfoque, no prohibiciones: encontrar defectos sigue siendo valioso.

21.2. Nueve principios y seis prácticas: conexión práctica

Los **nueve principios de la cátedra** son: testing temprano; testing no como fase; todos participan; feedback rápido; pruebas como expectativas; código limpio y defectos corregidos rápido; reducción de documentación innecesaria; pruebas como parte de Done; y desarrollo guiado por pruebas. Las **seis prácticas** son automatización unitaria y de integración, regresión automatizada a nivel de sistema, exploración, TDD, ATDD y control de versiones de pruebas junto con el código.

Ejemplo: una historia no está Done porque «el programador terminó». Debe superar los criterios acordados, incluidos los controles relevantes. Un cambio que rompe una prueba del pipeline debe corregirse oportunamente para evitar acumular incertidumbre. El tester puede colaborar antes de codificar, aportando ejemplos de casos inválidos y límites.

21.3. TDD y ATDD: un mismo producto, dos perspectivas

TDD: el programador escribe una prueba pequeña que falla, implementa lo necesario y refactoriza. **ATDD:** negocio, tester y desarrollador acuerdan pruebas y ejemplos de aceptación antes de implementar. Ejemplo: una historia sobre cancelar una compra utiliza ATDD para acordar hasta cuándo está permitida la cancelación, mientras TDD puede construir reglas unitarias que representen esas decisiones. Las dos prácticas son complementarias y no reemplazan pruebas exploratorias, de seguridad o rendimiento.

21.4. Los cuadrantes: evitar confundirlos con niveles cronológicos

Los **cuadrantes de testing ágil** organizan pruebas según su orientación **al negocio o a la tecnología** y su propósito de **guiar el desarrollo o evaluar/criticar el producto**.

Cuadrante	Perspectiva	Ejemplos	Para qué sirve
Q1	Tecnología / guía	Unitarias, componentes, TDD	Diseñar código y obtener feedback técnico rápido
Q2	Negocio / guía	Ejemplos funcionales, pruebas de historias, ATDD, prototipos	Aclarar y comprobar expectativas del cliente
Q3	Negocio / evaluación	Exploratorias, usabilidad, UAT, Alfa/Beta	Examinar el uso real y descubrir necesidades no previstas
Q4	Tecnología / evaluación	Rendimiento, carga, seguridad, recuperación	Examinar atributos técnicos de calidad y riesgos

No hay un orden temporal obligatorio Q1-Q2-Q3-Q4. Si la capacidad o seguridad es el mayor riesgo, conviene abordar Q4 tempranamente. Las pruebas de Q1 y Q2 apoyan la construcción; las de Q3 y Q4 examinan críticamente el producto. Todas pueden aportar información para la mejora de futuras historias.

21.5. Pirámide de automatización

La pirámide propone una **base amplia de pruebas unitarias y de componentes**, un **nivel intermedio de pruebas funcionales/servicios/API** y una **cima estrecha de pruebas GUI de extremo a extremo**. Las pruebas de interfaz son útiles para algunos recorridos completos, pero a menudo resultan más lentas y frágiles ante cambios de presentación. Esto explica la preferencia por numerosos tests rápidos en niveles inferiores. No hay una distribución porcentual universal que deban cumplir todos los proyectos.

Automatización no significa eliminar exploración. Los scripts ejecutan verificaciones conocidas; las actividades exploratorias, de usabilidad y aceptación de usuarios buscan aprender qué no se había previsto. La revisión de los resultados de tests y la mantenimiento de scripts también consumen esfuerzo.

21.6. El tester dentro del Sprint

Antes del desarrollo: refina historias con Product Owner y programadores; plantea ejemplos y criterios de aceptación. **Durante el desarrollo:** prepara datos y ambientes, automatiza o colabora con automatización, realiza exploración y comunica hallazgos. **Para declarar Done:** verifica que se satisfacen los acuerdos de prueba, calidad e integración. **En la revisión:** ayuda a obtener feedback de interesados. **En la retrospectiva:** presenta problemas del proceso, propone mejoras y analiza impedimentos recurrentes.

Una forma útil de colaborar es el **Power of Three** descrito por Crispin y Gregory: negocio, desarrollo y tester conversan sobre una historia para alcanzar comprensión compartida. No es un rol nuevo ni una ceremonia obligatoria de Scrum; es una práctica de comunicación.

21.7. Cambios culturales y condiciones de éxito

Una transición difícil es abandonar departamentos aislados, donde desarrollo «lanza» código a QA para que éste lo repare mediante pruebas tardías. El enfoque de **whole team** exige capacitación, comunicación y responsabilidad compartida. Un tester debe poder plantear riesgos sin convertirse en antagonista del programador, y un desarrollador debe asumir que verificar su propio trabajo es parte de construirlo.

Crispin y Gregory describen **siete factores de éxito**: equipo completo; mentalidad de testing ágil; automatizar regresión; proporcionar y obtener feedback; construir prácticas técnicas fundamentales; colaborar con clientes; mantener una visión global. Las prácticas fundamentales incluyen integración continua, ambientes de prueba confiables, control de deuda técnica, trabajo incremental y programación/testing como un solo proceso.

Errores típicos: afirmar que en Agile no se documenta ni se planifica; creer que todo debe automatizarse; suponer que solo el tester prueba; asumir que los cuadrantes son fases; interpretar Done como «código escrito»; y dedicar todas las pruebas de performance a la última semana.

22. Guía de repaso por preguntas: Unidad 4

¿Por qué no alcanza el testing para asegurar calidad? Porque detectar errores después no corrige necesidades mal comprendidas ni sustituye procesos de prevención, gestión y evaluación; las pruebas proporcionan información, no garantía absoluta.

¿Qué relación existe entre proceso y producto? Un proceso adecuado ayuda a lograr resultados repetibles y prevenir defectos, pero no garantiza que el producto satisfaga necesidades si éstas están mal definidas.

¿Dónde ubica la cátedra PPQA en CMMI? En CMMI-DEV v1.3, nivel de madurez 2, categoría Soporte.

¿Qué diferencia hay entre una auditoría y una inspección? La primera evalúa conformidad frente a criterios mediante evidencia objetiva; la segunda busca defectos en el contenido de un producto de trabajo.

¿Cuáles son las fases de la auditoría? Preparación y planificación; ejecución; análisis y reporte; seguimiento.

¿Qué diferencia hay entre auditoría funcional y física? La funcional contrasta el producto con requisitos; la física contrasta la configuración construida con su documentación.

¿Qué diferencia hay entre un plan y un caso de prueba? El plan organiza objetivos, riesgos, estrategia y recursos; el caso define entradas, pasos, condiciones y resultados esperados para una verificación concreta.

¿Por qué las pruebas deben diseñarse por riesgo? Porque la prueba exhaustiva no es viable y los recursos deben concentrarse donde un defecto tenga consecuencias relevantes.

¿Qué diferencia hay entre caja negra y caja blanca? Caja negra usa especificaciones y comportamiento observable; caja blanca utiliza estructuras internas del código.

¿Cuáles son las seis etapas de una inspección? Planificación, visión general, preparación, reunión, corrección y seguimiento.

¿Qué diferencia hay entre TDD y ATDD? TDD guía el diseño técnico mediante pruebas pequeñas; ATDD aclara comportamientos de negocio mediante ejemplos y pruebas de aceptación.

¿Por qué una pirámide con muchas pruebas GUI es problemática? Pueden ser lentas, costosas de mantener y frágiles ante cambios de interfaz; conviene distribuir las verificaciones hacia niveles adecuados.

¿Los cuatro cuadrantes se ejecutan en orden numérico? No. Son una clasificación de propósitos, y la oportunidad de cada prueba depende de riesgos y necesidades del producto.

Resumen para el parcial · Unidad 4

Calidad: satisface necesidades explícitas e implícitas; es relativa al usuario, el contexto y el tiempo. Cinco visiones: trascendental, producto, usuario, manufactura y valor. Calidad programada, realizada y necesaria deben coincidir. Calidad de proceso pregunta cómo se trabaja; calidad de producto, qué se obtuvo. ISO/IEC 25010:2011 describe ocho características de calidad de producto.

SQA: establecer y evaluar prácticas de calidad y reportar problemas; planificación define criterios del proyecto y QC comprueba su cumplimiento. Testing no equivale a toda la calidad. Verificación: ¿construimos correctamente? Validación: ¿construimos lo correcto? Calidad debe planificarse y ser responsabilidad compartida.

Modelos: CMMI-DEV v1.3, niveles de madurez **1 Inicial, 2 Administrado, 3 Definido, 4 Administrado cuantitativamente, 5 Optimizado**. **PPQA está en nivel 2, categoría Soporte**. SPICE clásico: capacidad **0 a 5** por proceso. ISO es una familia de normas; ISO/IEC 25010:2011 describe características del producto. IDEAL: iniciar, diagnosticar, establecer, actuar y aprender.

Auditoría: evaluación objetiva e independiente de conformidad con criterios. Roles: gerente SQA, auditor y auditado. **Cuatro fases: planificación, ejecución, análisis/reporte, seguimiento**. Resultados: buena práctica, desviación, observación. Auditoría de proyecto examina proceso y consistencia; funcional contrasta software y ERS; física contrasta software y documentación.

Testing: Test Plan contiene riesgos y objetivos, estrategia, recursos y criterios de aceptación. Niveles: **unitario, integración, sistema, aceptación**. Tipos: funcionales, no funcionales, regresión, smoke, etc. Caja negra: equivalencias, límites, tablas, casos de uso, estados; caja blanca: coberturas; experiencia: error guessing y exploratorio. La ausencia de defectos observados no demuestra ausencia absoluta de defectos.

Revisiones técnicas: estáticas. **Walkthrough** informal con feedback rápido; **inspección** formal y medible. Roles: autor, moderador, lector, anotador, inspector. Etapas: **planificación, visión general, preparación, reunión, corrección, seguimiento**. Revisar producto, no productor.

Testing ágil: integrado a todo el desarrollo, responsabilidad compartida, feedback temprano y pruebas como parte del Done. Seis prácticas: automatización unitaria/integración, regresión, exploración, TDD, ATDD, control de versión de pruebas. **Q1 tecnología-guía; Q2 negocio-guía; Q3 negocio-evalúa; Q4 tecnología-evalúa.** La pirámide propone muchas unitarias, pruebas intermedias de API y pocas GUI automatizadas. Los cuadrantes no son fases.

MAPA DE COBERTURA PARA ESTUDIAR SIN DEJAR TEMAS FUERA

Este apartado sirve para comprobar que los puntos pedidos en la conversación tienen un desarrollo dentro de la guía. Se refiere al **temario trabajado en las respuestas** y no pretende sustituir la planificación oficial completa de la asignatura.

Unidad y tema pedido	Dónde encontrarlo
U2: concepto, origen y filosofía Lean	U2, apartados 1 y 5
U2: siete principios de Poppendieck, siete desperdicios y VSM	U2, apartados 1 y 5
U2: Kanban en software, principios, valores y seis prácticas	U2, apartados 2 y 6
U2: WIP, push/pull, políticas, clases de servicio, STATIK y cadencias	U2, apartados 2 y 6
U2: lead time, cycle time, throughput, WIP, Ley de Little, CFD y SLE	U2, apartados 3 y 7
U2: métricas Tradicional/Lean/Agile: EVM, velocidad, burndown y otras	U2, apartados 4 y 7
U3: concepto de prácticas continuas y relación con producto	U3, apartados 1 y 7
U3: integración continua, flujo y riesgos de integración tardía	U3, apartados 2 y 7
U3: Private System Build, Integration Build y Release Build	U3, apartados 3 y 7
U3: testing continuo, TDD y pirámide de automatización	U3, apartados 4 y 7
U3: Continuous Delivery y Continuous Deployment	U3, apartados 5 y 7; identificados como complemento técnico
U3: SCM, feedback, releases y ejemplo integrador	U3, apartados 6 y 7
U4: concepto de calidad, cinco visiones, tres dimensiones	U4, apartados 1 y 16
U4: calidad de proceso y de producto, ISO 25010:2011	U4, apartados 2 y 16
U4: importancia, ventajas, costos, dificultades y diez principios	U4, apartados 3 y 16
U4: QA, planificación, QC, testing, verificación y validación	U4, apartados 4, 5 y 16
U4: actividades de SQA y Grupo de Aseguramiento de Calidad	U4, apartados 5 y 16
U4: CMMI-DEV v1.3, SPICE, normas ISO e IDEAL	U4, apartados 6 y 17
U4: auditorías de proyecto, configuración funcional y física	U4, apartados 7 y 18
U4: proceso de auditoría, roles, checklist, informe y seguimiento	U4, apartados 7 y 18

Unidad y tema pedido	Dónde encontrarlo
U4: auditorías al Grupo de Calidad	U4, apartados 7.6 y 18.5, con aclaración de alcance bibliográfico
U4: planificación y proceso del testing, Test Plan y casos de prueba	U4, apartados 8 y 19
U4: niveles, tipos y técnicas de pruebas; herramientas	U4, apartados 9 a 13 y 19
U4: revisiones técnicas, walkthrough e inspecciones de Fagan	U4, apartados 14 y 20
U4: testing ágil, nueve principios, TDD/ATDD, cuadrantes y pirámide	U4, apartados 15 y 21

Cómo usar la guía: primero leé cada unidad en orden, después resolvé los ejemplos sin mirar la explicación y finalmente usá el resumen al cierre de cada unidad. Si estás repasando para una pregunta oral, intentá explicar qué problema resuelve cada práctica, cómo funciona, qué información necesita y con qué conceptos suele confundirse.

Nota de fidelidad: las definiciones y versiones se ajustan al material de la cátedra cuando éste las desarrolla. Los complementos técnicos señalados —por ejemplo, Continuous Delivery/Deployment o niveles del SPICE clásico— sirven para completar el razonamiento, pero no deben presentarse como transcripciones de esas diapositivas.

Bibliografía de referencia

Material principal de la cátedra: UTN-FRC, Judith Meles y Laura Covaro, presentaciones *Testing en Ambientes Ágiles*; *Aseguramiento de Calidad de Proceso y Producto (PPQA)*; *Revisiones Técnicas*; *Auditorías de Software*, todas incluidas en el PDF compilado `ilovepdf_merged (5).pdf`.

Lean y Kanban: Mary y Tom Poppendieck, *Lean Software Development: An Agile Toolkit*; *Implementing Lean Software Development: From Concept to Cash*. David J. Anderson, *Kanban: Cambio Evolutivo Exitoso*. Anderson y Carmichael, *Kanban Esencial Condensado*. *Guía oficial del Método Kanban*. Kniberg y Skarin, *Kanban y Scrum: obteniendo lo mejor de ambos*. Gothelf y Seiden, *Lean UX*. Mike Cohn, *Agile Estimating and Planning*.

Prácticas continuas y Scrum/XP: Berczuk, Appleton y Konieczka, *Agile SCM - Build Management for an Agile Team*. Henrik Kniberg, *Scrum y XP desde las trincheras*.

Testing y calidad: Lisa Crispin y Janet Gregory, *Agile Testing: A Practical Guide for Testers and Agile Teams*. Federico Toledo y colaboradores, *Introducción a las Pruebas de Sistemas de Información*. Glenford J. Myers y colaboradores, *The Art of Software Testing*. Jorgensen, *Software Testing: A Craftsman's Approach*.

Nota de fidelidad: se conservaron los modelos, denominaciones y versiones enseñados en el material aportado. Algunas explicaciones complementarias (Continuous Delivery frente a Continuous Deployment, niveles clásicos de SPICE y normas ISO de contexto) no aparecen con idéntico desarrollo en las diapositivas y fueron diferenciadas de los contenidos textuales de la cátedra.